

TEMA 4

INTRODUCCIÓN A LA SEGURIDAD DE APLICACIONES



Creando código seguro



"S-82 NARCISO MONTURIOL" PROJECT v5.0

Por favor, informa de problemas con los materiales a redondojose@uniovi.es

Logo: @creative_vanesa (Instagram)



Departamento de Informática
Universidad de Oviedo



Universidad de Oviedo



ÍNDICE

Fundamentos de Seguridad de Aplicaciones

Seguridad a Nivel de Requisitos

 Manejo de datos

 Autenticación

 Otros

Diseño Seguro

 Threat modeling

Creación de Código Seguro

 Autenticación y autorización

 Log y errores

Application Security Testing (AST)

 Herramientas estáticas

 Herramientas dinámicas

Despliegue y Mantenimiento Seguros

[< Ir al Índice](#)



FUNDAMENTOS DE SEGURIDAD DE APLICACIONES

Conceptos básicos antes de empezar

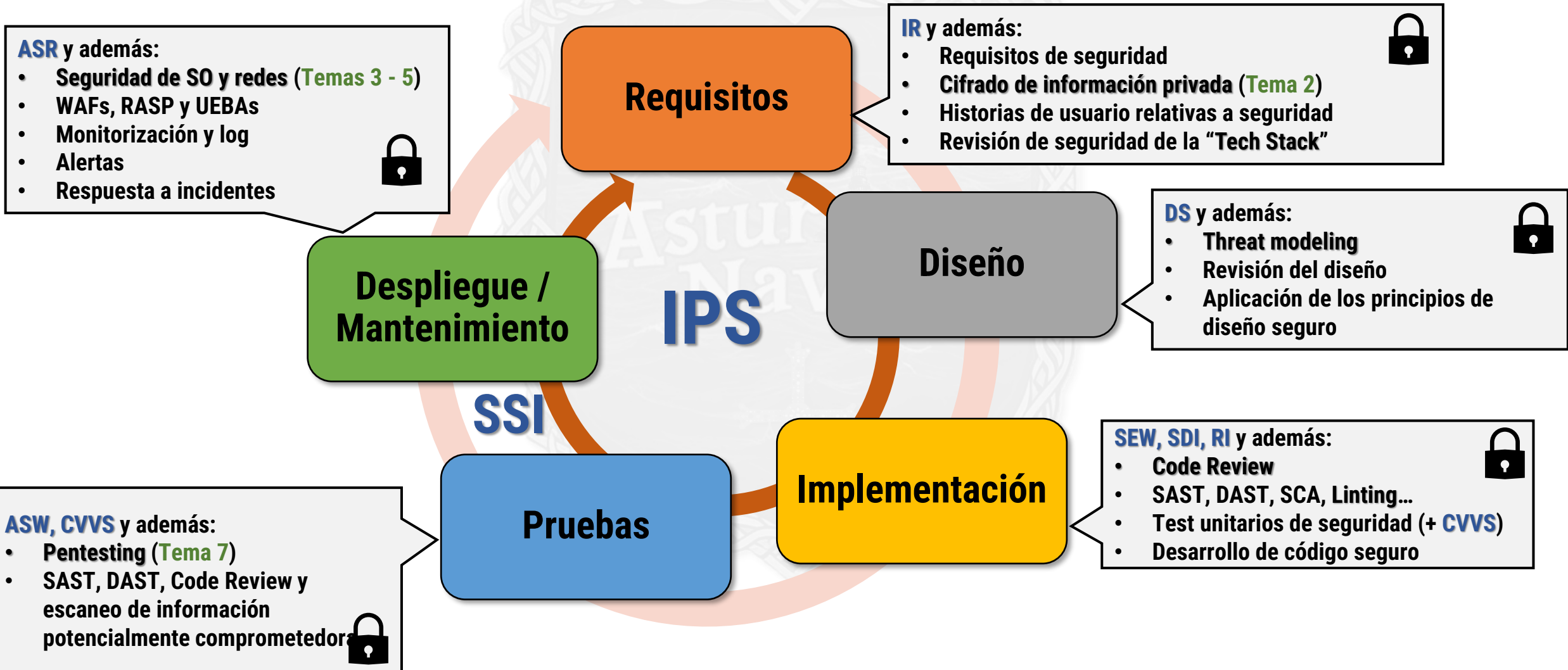


Departamento de Informática
Universidad de Oviedo



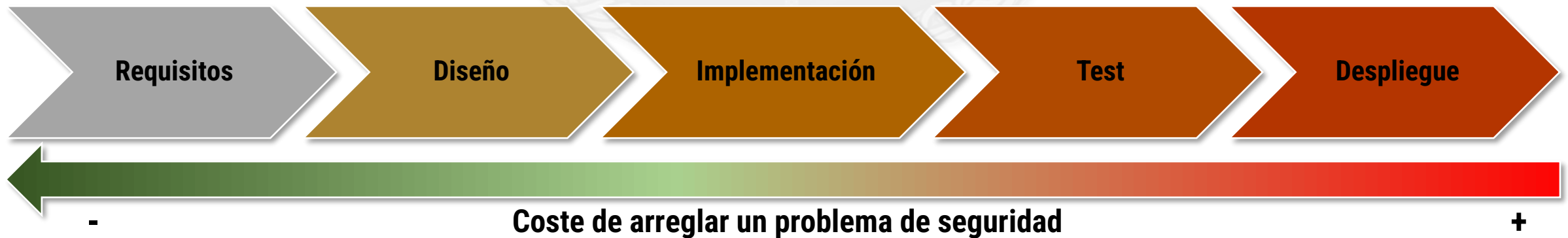
Universidad de Oviedo

DEL SDLC AL S-SDLC (TERCER CURSO, INTEGRADO Y "SECURIZADO")



LA FILOSOFÍA “PUSHING LEFT”

- Cuanto más tarde se descubra un problema de seguridad en el S-SDLC, **más caro** será solucionarlo (en tiempo y recursos)
- Todas las características de seguridad de la aplicación deben resolverse lo antes posible en la cadena SDLC
 - A esto se le suele denominar **filosofía “pushing left”**
 - Por ello, en este tema la mayoría de los contenidos **están en las fases iniciales de SDLC**



[< Ir al Índice](#)



SEGURIDAD A NIVEL DE REQUISITOS

La seguridad se planifica desde el minuto cero



[Manejo de datos >](#)

[Autenticación >](#)

[Otros >](#)



Departamento de Informática
Universidad de Oviedo



Universidad de Oviedo

¿QUÉ TE VAS A ENCONTRARTE EN ESTE BLOQUE?

● En la primera sección de este bloque aprenderás

- La importancia de **tratar cualquier entrada que recibe la aplicación como insegura**, y por qué es imprescindible su validación
 - Y también los distintos **puntos donde puedes tener entrada de datos**
- La importancia (y diferencia) entre **codificación y "escapado"** de caracteres
- Lo que debes hacer para **crear cookies seguras** en tu aplicación web

● En la segunda, aprenderás

- Las claves para una **autenticación segura**: Requisitos de gestión de contraseñas y MFA
- La diferencia entre una **cuenta de servicio y de usuario** y por qué las primeras son mejores para usarlas con aplicaciones

● Y en la tercera aprenderás

- Lo enormemente importante que es **minimizar el uso de componentes de terceros**
- **Otros requisitos importantes de seguridad**: Cabeceras HTTP, backups, subida de ficheros, logs, formas de autenticación y autorización, mínimo privilegio, etc.



PREGUNTAS DE SEGURIDAD EN LOS REQUISITOS

- En 4º curso hay una asignatura llamada **Ingeniería de Requisitos (IR)**
 - Una persona o equipo debe desarrollar los requisitos de seguridad de la aplicación
 - Y añadir nuestros requisitos de seguridad a los que obtengas en esa asignatura
 - No importa tu metodología de desarrollo (**IPS, ASW**), lenguaje (**TPP**) o framework (**SDI, SEW**)
- Deben hacer preguntas con las que se elaborarán los requisitos
 - *¿Contiene o maneja información confidencial, sensible o que identifica a las personas (Personal Identifiable Information, PII)?*
 - *¿Dónde y cómo se guardan los datos?*
 - *¿Está disponible públicamente (Internet)? ¿O no? (solo para una Intranet)*
 - *¿Va la aplicación a realizar tareas sensibles o esenciales?*
 - Transferencias de dinero, manejo de datos médicos...
 - *¿Hace algo considerado potencialmente peligroso?*
 - Ej.: Permitir a los usuarios subir ficheros, manejar datos de pago, tener varios roles...
 - ... (¡No te cortes!, “Arrasa con lo que veas, y generoso no seas”)
 - <https://simeio.com/resources/infographic/infographic-the-6-identity-and-access-security-questions-you-need-to-answer>



Piensa mal...¡y
(literalmente) acertarás!

EL APPLICATION SECURITY VERIFICATION STANDARD (ASVS)



- Pero te puedes encontrar con dos problemas
 - No saber cómo **empezar** adecuadamente por falta de experiencia
 - Lo mismo que los **controles de seguridad del Tema 3**: Necesitas una lista de requisitos **verificable** y **certificada**
- En ambos casos podemos usar el **estándar ASVS de OWASP**
 - <https://owasp.org/www-project-application-security-verification-standard/>
 - O mejor en **CSV**, para llevar mejor la contabilidad
- Este estándar es muy útil para verificar la seguridad de aplicaciones, para arquitectos y también desarrolladores
 - Qué partes son de **acceso abierto** y cuáles **restringidas**
 - Mejorar la **documentación** del proyecto
 - **Controles de seguridad** a implementar en el código fuente
 - ...

NIVELES DE CUMPLIMIENTO ASVS



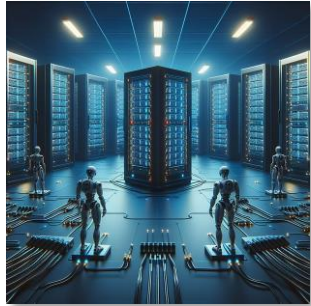
● Determinan qué requisitos utilizamos

- **Nivel 1 (Mínimo):** Requisitos de seguridad básicos
 - Pretende ser un paso intermedio para alcanzar el Nivel 2
 - Creado para facilitar la implementación del ASVS
 - ¡Adecuado para tu TFG!
- **Nivel 2 (Estándar): Nivel deseable** para la mayoría de las aplicaciones
 - Defienden adecuadamente contra **la mayoría de los riesgos** asociados al software actual
- **Nivel 3 (Avanzado):** Para **aplicaciones de alto valor**, alta seguridad o críticas para la función
 - Ej.: Militar, sanidad, seguros, infraestructuras críticas...
 - Requiere un **análisis profundo** de la arquitectura, el código y las pruebas
 - Puede ser **excesivo** para aplicaciones típicas; ¿Por qué?

● Vamos a detallar algunos de estos requisitos

chapter_id	chapter_name	section	section_name	req	req_description	Level
V1	Encoding and Sanitization	V1.1	Encoding and Sanitization Architecture	V1.1.1	Verify that input is decoded or unescaped into a canonical form only once, it is only decoded when encoded data in that form is expected, and that this is done before processing the input further, for example it is not performed after input validation or sanitization.	2
V1	Encoding and Sanitization	V1.1	Encoding and Sanitization Architecture	V1.1.2	Verify that the application performs output encoding and escaping either as a final step before being used by the interpreter for which it is intended or by the interpreter itself.	2
V1	Encoding and Sanitization	V1.2	Injection Prevention	V1.2.1	Verify that output encoding for an HTTP response, HTML document, or XML document is relevant for the context required, such as encoding the relevant characters for HTML elements, HTML attributes, HTML comments, CSS, or HTTP header fields, to avoid changing the message or document structure.	1
V1	Encoding and Sanitization	V1.2	Injection Prevention	V1.2.2	Verify that when dynamically building URLs, untrusted data is encoded according to its context (e.g., URL encoding or base64url encoding for query or path parameters). Ensure that only safe URL protocols are permitted (e.g., disallow javascript: or data:).	1
V1	Encoding and Sanitization	V1.2	Injection Prevention	V1.2.3	Verify that output encoding or escaping is used when dynamically building JavaScript content (including JSON), to avoid changing the message or document structure (to avoid JavaScript and JSON injection).	1
V1	Encoding and Sanitization	V1.2	Injection Prevention	V1.2.4	Verify that data selection or database queries (e.g., SQL, HQL, NoSQL, Cypher) use parameterized queries, ORMs, entity frameworks, or are otherwise protected from SQL injection and other database injection attacks. This is also relevant when writing stored procedures.	1
V1	Encoding and Sanitization	V1.2	Injection Prevention	V1.2.5	Verify that the application protects against OS command injection and that operating system calls use parameterized OS queries or use contextual command line output encoding.	1
V1	Encoding and Sanitization	V1.2	Injection Prevention	V1.2.6	Verify that the application protects against LDAP injection vulnerabilities, or that specific security controls to prevent LDAP injection have been implemented.	2
V1	Encoding and Sanitization	V1.2	Injection Prevention	V1.2.7	Verify that the application is protected against XPath injection attacks by using query parameterization or precompiled queries.	2
V1	Encoding and Sanitization	V1.2	Injection Prevention	V1.2.8	Verify that LaTeX processors are configured securely (such as not using the "-shell-escape" flag) and an allowlist of commands is used to prevent LaTeX injection attacks.	2
V1	Encoding and Sanitization	V1.2	Injection Prevention	V1.2.9	Verify that the application escapes special characters in regular expressions (typically using a backslash) to prevent them from being misinterpreted as metacharacters.	2
V1	Encoding and Sanitization	V1.2	Injection Prevention	V1.2.10	Verify that the application is protected against CSV and Formula injection. The application must follow the escaping rules defined in RFC 4180 sections 2.6 and 2.7 when exporting CSV content. Additionally, when exporting to CSV or other spreadsheet formats (such as XLS, XLSX, or ODF), special characters (including '=', '*', ';', '@', '\r' (tab), and '\0' (null character)) must be escaped with a single quote if they appear as the first character in a field value.	3
V1	Encoding and Sanitization	V1.3	Sanitization	V1.3.1	Verify that all untrusted HTML input from WYSIWYG editors or similar is sanitized using a well-known and secure HTML sanitization library or framework feature.	1
V1	Encoding and Sanitization	V1.3	Sanitization	V1.3.2	Verify that the application avoids the use of eval() or other dynamic code execution features such as Spring Expression Language (SpEL). Where there is no alternative, any user input being included must be sanitized before being executed.	1

Si quieres usar esto (para tu TFG ;) ;)), te ofrecemos un Excel para filtrarlos y gestionarlos fácilmente



1 2
3 4

Requisitos de manejo de datos

Distintas técnicas para tratar tus datos de forma segura



REQUISITOS DE MANEJO DE DATOS

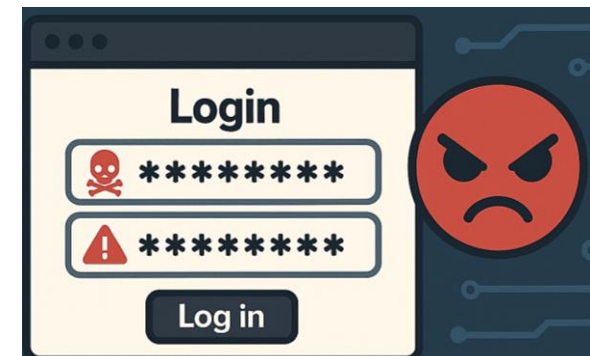


- **Nunca confíes en la entrada del usuario**
 - **Toda entrada** que llegue al sistema puede ser **falsificada** o manipulada **maliciosamente**
 - La aplicación puede fallar de formas no previstas y se vuelve inestable
 - Por tanto, **TODA ENTRADA debe ser validada** y manejada adecuadamente
 - Siempre hay que **basarse en listas de entradas admitidas** si es posible
 - Usa además reglas de validación verificadas:
https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
 - **Valida siempre en el lado del servidor**
 - Validar en el cliente SÓLO en JavaScript **no es útil para la seguridad**; ¿Por qué? (Seminario 3)
- **Una entrada NO es fiable hasta que ha superado tus procedimientos de validación**
 - Validación **sintáctica** y **semántica** (la entrada **es adecuada y tiene el tipo correcto**); ¿Significado?
 - Cualquier discrepancia debe causar el **rechazo de la entrada**; ¿Por qué?
- **¿Pero si se sabe cómo y dónde hacerlo, cuál es el problema?**
 - Que muchas veces no sabemos la **ENORME CANTIDAD** de fuentes de datos que manejamos

POSIBLES PUNTOS DE ENTRADA DE DATOS

● Todas las entradas deben validarse

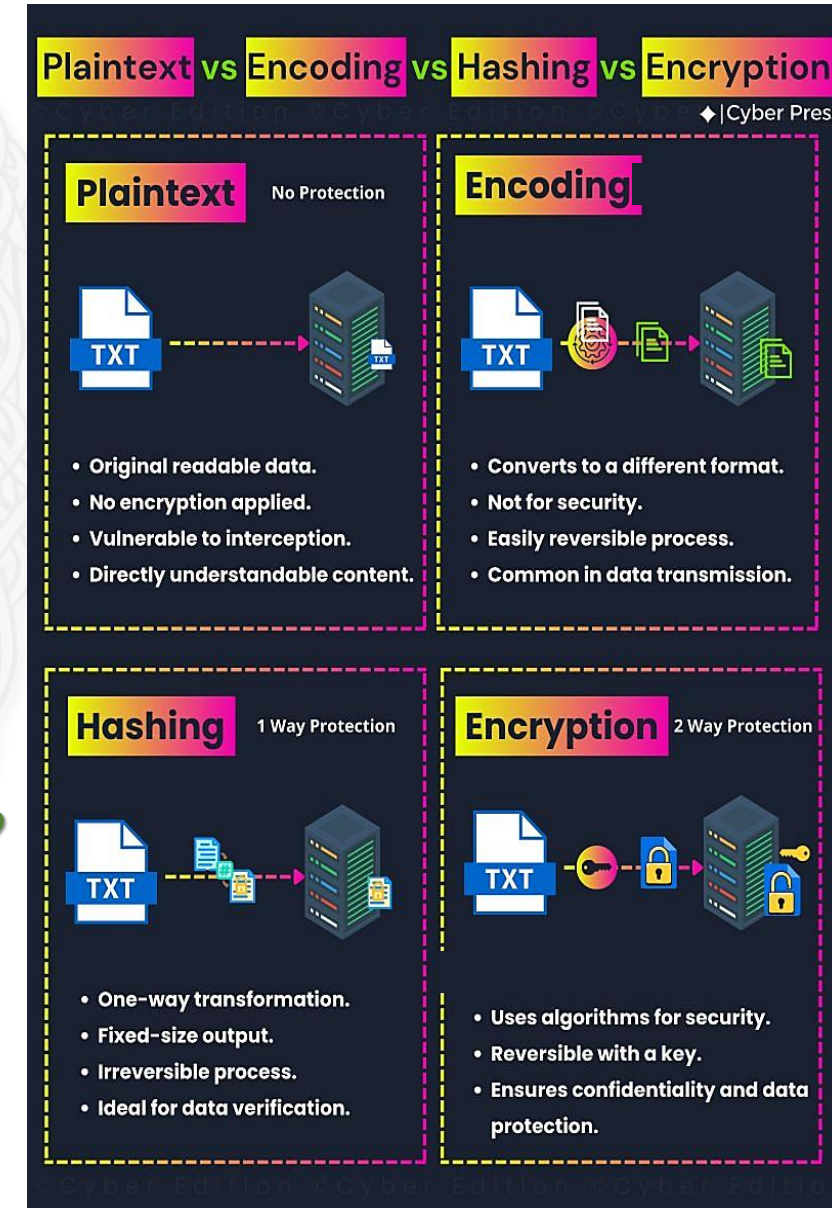
- **Cualquier entrada del usuario** (la obvia), incluyendo cabeceras HTTP, campos “hidden”
- Información que se reciba de
 - **SGBDs** (sean tuyos o no)
 - **APIs** (incluso si la has hecho tú mismo)
 - Código incluido en la aplicación y compilado con ella (**dependencias de terceros**)
 - Bibliotecas, frameworks, código de GitHub... (**supply-chain attack**)
 - **De otra aplicación:** Especialmente si la nuestra acepta entradas de ella
 - Incluye aplicaciones **serverless**, **scripts**, **frameworks** externos, información para el log ...
 - **Valores en la URL** (parámetros), en las **cookies**, en los ficheros de configuración...
 - Especialmente si las **cookies** no tienen los **flags** “**secure**” y “**HTTPS only**”
 - Incluye valores que se usen de sistemas de almacenamiento online (S3, Firebase...)
 - Sé especialmente cuidadoso con las **URLs** o los **ficheros subidos**; ¿**Por qué?**
 - https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
- Datos o **comandos** que vengan de aplicaciones en nube



REQUISITOS DE GESTIÓN DE DATOS

● Otros requisitos de gestión de datos

- **Privacidad de datos:** Política legal que especifique los datos recogidos por cookies ([ASLEPI](#))
- **Clasificación de datos:** Y **etiquetado** por su privacidad
 - Depende del **país o legislación aplicable**
 - **Por ejemplo:** Classified, Secret, Top Secret ([ASLEPI](#))
 - "Guide for Mapping Types of Information and Information Systems to Security Categories": <https://www.nist.gov/publications/guide-mapping-types-information-and-information-systems-security-categories-2-vols>
 - Determina **cómo se almacena** (*¿Cifrado? ¿Hash?*)
- **Transformación de datos:** Evita ataques de "inyección"; *¿Qué son?*
 - "Escapar": Eliminar "el poder" que tiene en un contexto; *¿Qué significa esto?*
 - **Codificación: Tema 2**
 - El formato objetivo es **inofensivo** en el contexto que se va a usar



● Debemos recordar usar algoritmos de cifrado fuertes y... (Tema 2)

- Las contraseñas e información sensible **NUNCA** se guardan en texto plano
- Se usan **técnicas de hashing** para las contraseñas (Key Derivation Functions)
 - Y cualquier información **cuyo valor original no sea necesario**; *¿Por qué?*
 - Identidad, autenticación, integridad de los datos...
- La **información sensible** debe estar cifrada
 - **En tránsito**: usuario <-> frontend, frontend <-> backend, backend <-> SGBD, API <-> API, etc. (Ej.: HTTPs)
 - **En reposo**: Cuando se guarda
 - *¿Esto garantiza la CIA?*
- Cifrar datos en RAM **puede ser excesivo** (ASVS Nivel 3)

Р.Дх%бК,.4щ.Зь.э.)·ЕУ.м..ШяUD9В`im^уиЦэ-Я.ШьјЙ]Ту2Ньw&кЗ_ьВ.Сйμ#Е‡>.чВ>*г...Щ'
_ФхSE..Уf.ШSfииУ.°ZuP\$SЙGэБННцг..J@xpX.. j]Вд5ю=LG?...OnX_КПЗ.]¶>НьХg^..ььВК
}ь1ь@*ВЗ¶-.Unŷ.. С".у»..JvHg<ньа».μSje-.OP|d.г<ш·.сFЭИ..'.Иь·ч,.6о у.ЇкЗным<
НУ.БР'у.Ј...Ш.1к·:2Ньш.*Т.-Кл.Ш¶у"С1Ш"ГВ.мүФ0Е-.ngsHr.с"ХьГБ}..zЕа.9ецЖ0ШzчПь
Хч7дНш.Ей-"..hPrмюУUшр..хэп..W6ьε9х.пдg)CGoЮ»0т.3.}mdESAG·.r»..@У...Wf-.'јф
З->{(vfфында s.Т(бц..]2}ГSjX8ФъщыШЙ|vzНьж2.ё]уЗЪ9Ъ·Вмд[ЩГШ|р`[h"Гк"ЗЦфёŷ~Jhq.
·'ЖИС.РнебУfi»q5D·XBё[-~,g.мN1\$ЛуЦ7~.\\ы нцую<.=ефЧ|..у<.¶sП|!Ньг8-бш*Т~ьтЮ
μIvШпЁхү.'?ФЗŷJ%=К·..ь..Gщd·X;іЪХε_Б.ВфюШь~.ш+..Wщ+.к·хьВ'нФк...ШУК]»іЪ-ЇG
"вЁ=ньМ,ньУ;·с.щ.Нь|х·М.1Z¶ЙС1Цщ±-.уХіУя\$0ькГ'.`чŷЕзлищ.@°щ=јЁ...}.FPAЙz#х 5`п
%|Тпийща..Тф.і...±...о="я/ђТ.;й2Г.Ф¶*.~Be&ε·З"х..ШУхhhнл.ёНЕэ.н.Й.щНьпхЗ:К.`
ІТуЈ»}чЪНУ`°.UE..&Gм0ŷ...УЯ*ŷ{.We@‡"Й-:БЇу.]·НьF@НШ4ЕЙсНь./....GDM."0εЧL..%

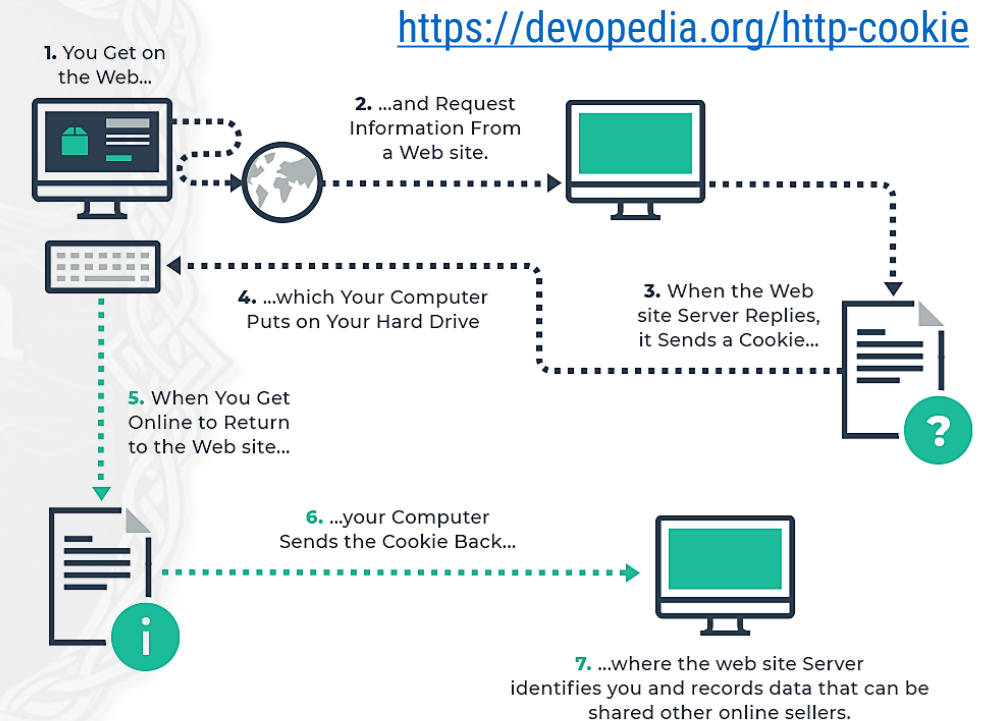
¡Los usuarios no autorizados deberían ver esto! ☺

REQUISITOS DE SEGURIDAD EN COOKIES

● Implementación del estado en HTTP (**SEW**)

● Sus requisitos son

- Cada uno de sus valores **debe ser validado** antes de usarse (son otra fuente de entrada)
 - *¿Crees que tiene sentido escribir información privada de una app en una cookie?*
- El ID de sesión debe SIEMPRE pasarse **en una cookie de sesión** (no persistente); *¿Por qué?*
- Debe **enviarse bajo conexiones cifradas** (flag Secure, **Set-Cookie: Secure**)
- No deben ser accesibles desde JavaScript (flag HTTPOnly, **Set-Cookie: HttpOnly**)
 - Esto evita algunos vectores de ataque XSS
- Deberían caducar (flag **Expires**)



Las cookies no tienen las mismas normas de seguridad que el "Almacenamiento Local" del navegador. Fuente:
https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html

REQUISITOS DE SEGURIDAD EN COOKIES



- **Domain** : Por defecto, las cookies de un dominio solo pueden ser accedidas por sitios web de ese dominio
 - Es seguro por defecto
 - Si queremos que otros dominios lean nuestras cookies, podemos hacerlo. Ej.: **Set-Cookie: Domain: ads.redondo.com**
 - Pero evítalo siempre que sea posible
- **Path**: Muchas aplicaciones están compuestas por diferentes subaplicaciones bajo una única URL base
 - Cada subaplicación posee un camino particular de la URL base
 - Podemos evitar que las cookies de una parte sean leídas desde otras (**Set-Cookie: path=/accounting**)
- **Same-Site**: Medida contra ataques CSRF (**Tema 6**)
 - El valor **Strict** (recomendado) significa que las cookies solo pueden provenir del mismo sitio que las utiliza
- Prefijos de cookies: Defensa en profundidad. Ej.: **__Secure**)
 - Para que solo ciertos subdominios puedan acceder a una cookie
 - <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie>

```
Set-Cookie: __Secure-  
SessionID=a4b3c2d1e0f9g8h7i6j5k4l3m  
2n1o0p;  
Domain=miempresa.com;  
Path=/  
Max-Age=3600;  
Secure;  
HttpOnly;  
SameSite=Strict
```

Ejemplo típico de una cookie de una aplicación que se ha tomado en serio todos los requisitos de seguridad para cookies 😊

¿CREEES QUE LO HAS ENTENDIDO TODO? ¡AUTOEVALÚATE!



● Eres capaz de entender lo siguiente

- *Que para pensar en requisitos de seguridad hay que hacer las preguntas adecuadas*
- *La diferencia entre cifrado en tránsito y cifrado en reposo*
- **Sobre requisitos de manejo de datos**
 - *Que ninguna entrada de usuario es confiable, en ninguna circunstancia*
 - *Que la entrada de usuario no son solo el contenido de los componentes **textfield** sino, además, muchas otras cosas relativas al protocolo HTTP, **software** de 3ºs con el que interaccionemos...*
 - *Que el tipo de datos esperado es algo clave a la hora de validarlos*
 - *Que los datos se deben etiquetar según su privacidad y la legislación aplicable*
 - *La diferencia entre codificar y escapar un dato, cuándo y para qué se usan*
 - *La función de todas las herramientas de seguridad de las cookies: **Secure, HttpOnly, Expires, Domain, Path** y **Same-Site***



Requisitos de autenticación

Lidiando con passwords e identidades de usuario



SECRETOS DE APLICACIONES Y CUENTAS

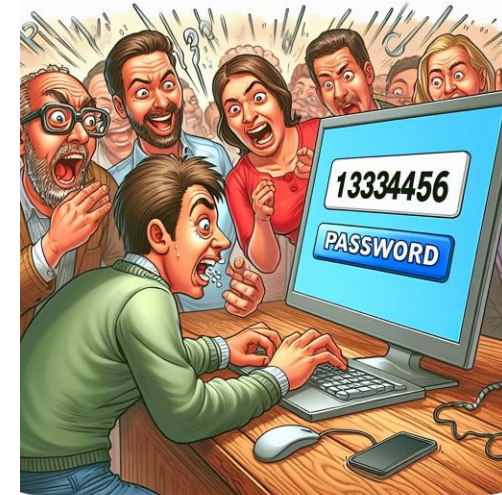


- Las **secret stores** son como password managers, pero para software, no personas
 - Es un archivo (“vault”) que guarda **secretos cifrados** (**Tema 2**) de una aplicación
 - *Desde la perspectiva del desarrollo, ¿Qué datos de una aplicación pondrías en un vault?*
 - **Acceso programático**: Desde el código de aplicación o desde el build de un pipeline CI/CD (**ASW**)
 - A nivel de requisitos, debe determinarse **QUÉ DATOS** deben ir en estas stores
 - **Es lo que DEBE hacerse en lugar del “hardcoding” de secretos de la aplicación**
 - Qué pasa si no lo usamos lo veremos en el **Seminario 4** (reversing)
- Cada aplicación debe funcionar bajo su propia identidad en una **cuenta de servicio**
 - Ej.: Una aplicación accede a archivos / bases de datos con permisos del usuario “SalesApp”
 - Este usuario no tiene permiso de inicio de sesión interactivo; *¿Qué significa esto?*
 - *¿Cuál es la principal ventaja de seguridad de hacer esto?*



REQUISITOS PARA PASSWORDS DE USUARIO

- Intenta **NO** guardar passwords en tu aplicación
 - Usa un servicio de autenticación de terceros si es posible (Google, Outlook...)
- Si no es posible, guarda claves como se indicó en el **Tema 2**
 - Usar un KDF, largas, complejas, **salt** (28+ caracteres), **pepper**, **strong hashing**...
- Añade además estos requisitos
 - **Fugas:** Usa APIs como **haveibeenpwned** para comprobar que una nueva clave no ha sido ya comprometida
 - <https://haveibeenpwned.com/API/v2>
 - **Nunca informar al usuario** de si lo que ha introducido mal es el nombre de usuario o la clave ; *¿Cuál es la mejor opción?*
 - No permitir **reutilizar / nuevas claves muy parecidas** ("Pass1", "Pass2", etc.)
 - **No hacer cambios periódicos forzosos**, solo cuando se descubra una brecha
 - **Usar MFA para todas las cuentas** (a continuación)



SECOND / MULTI-FACTOR AUTHENTICATION (2FA / MFA)

- Soportado por la alianza FIDO: <https://fidoalliance.org/>
 - Una alianza para **reducir la dependencia de las contraseñas a nivel mundial**
- Exige a los usuarios verificar su identidad de 2 (o más) formas únicas e independientes
 - Amplía el clásico usuario/contraseña con **un paso adicional de autenticación**
 - Normalmente requieren un token de un solo uso generado dinámicamente o una **contraseña de un solo uso (one-time password, OTP)**
 - **Solo el usuario adecuado debe saber el OTP:** Tiene que ser difícil de obtener para los atacantes
- Los MFA son obligatorios, dada la debilidad de las contraseñas
 - *¿Qué debilidades típicas tienen las contraseñas?*
 - *¿El MFA impide el acceso a una cuenta, aunque sepas la contraseña correcta?*
 - *¿Puede un MFA alertar a los usuarios de un robo de contraseñas?*



TIPOS DE MFA. PRINCIPIOS BÁSICOS

- Los esquemas MFA se basan en que los usuarios introduzcan 2 de 3 “algos”
 - Algo que **Saben**, como una **clave o PIN** de una cuenta
 - Algo que **Tienen**, como un dispositivo físico (**móvil**) o una aplicación que genera OTPs
 - Algo que **Son**, una característica biológica única como una **huella**, la **voz** o la **retina**
- Hay muchas formas de implementar MFAs
 - Incluso las peores aumentan la seguridad de las cuentas frente al simple acceso con contraseña



TIPOS DE MFA

- Como requisito, solo se deben usar niveles 6+ en la escala CASMM v6
- Ejemplos de métodos MFA seguros (recomendados) son
 - **Token hardware** : Ej.: YubiKey
 - **Token software**: Ej.: Google/Microsoft Authenticator, Steam Guard...
 - **Verificación biométrica**: El usuario es el token
 - Ej.: Lectores de huellas dactilares en teléfonos móviles

THE CONSUMER AUTHENTICATION STRENGTH MATURITY MODEL (CASMM) V6



Fuente: <https://danielmiessler.com/p/casmm-consumer-authentication-security-maturity-model>

¿CREEES QUE LO HAS ENTENDIDO TODO? ¡AUTOEVALÚATE!



● Eres capaz de entender lo siguiente

- *La diferencia entre un gestor de contraseñas y una secret store, para qué se usan y quién lo usa*
- *Qué política es la mejor a la hora de gestionar contraseñas y qué debes hacer si tienes que hacerlo tú mismo*
- *Qué es un MFA, sus tipos (con sus ventajas y desventajas), y cuál es el mejor desde el punto de vista de coste de implementación / seguridad*
- *Qué es una cuenta de servicio, y cómo y para qué debemos usarla en nuestras aplicaciones*
- *Cómo plantear los requisitos de un servicio de funcionalidad de contraseña olvidada*



Seguridad a nivel de requisitos



Otros requisitos de seguridad

Cosas adicionales que debemos considerar



REQUISITOS DE COMPONENTES DE TERCEROS

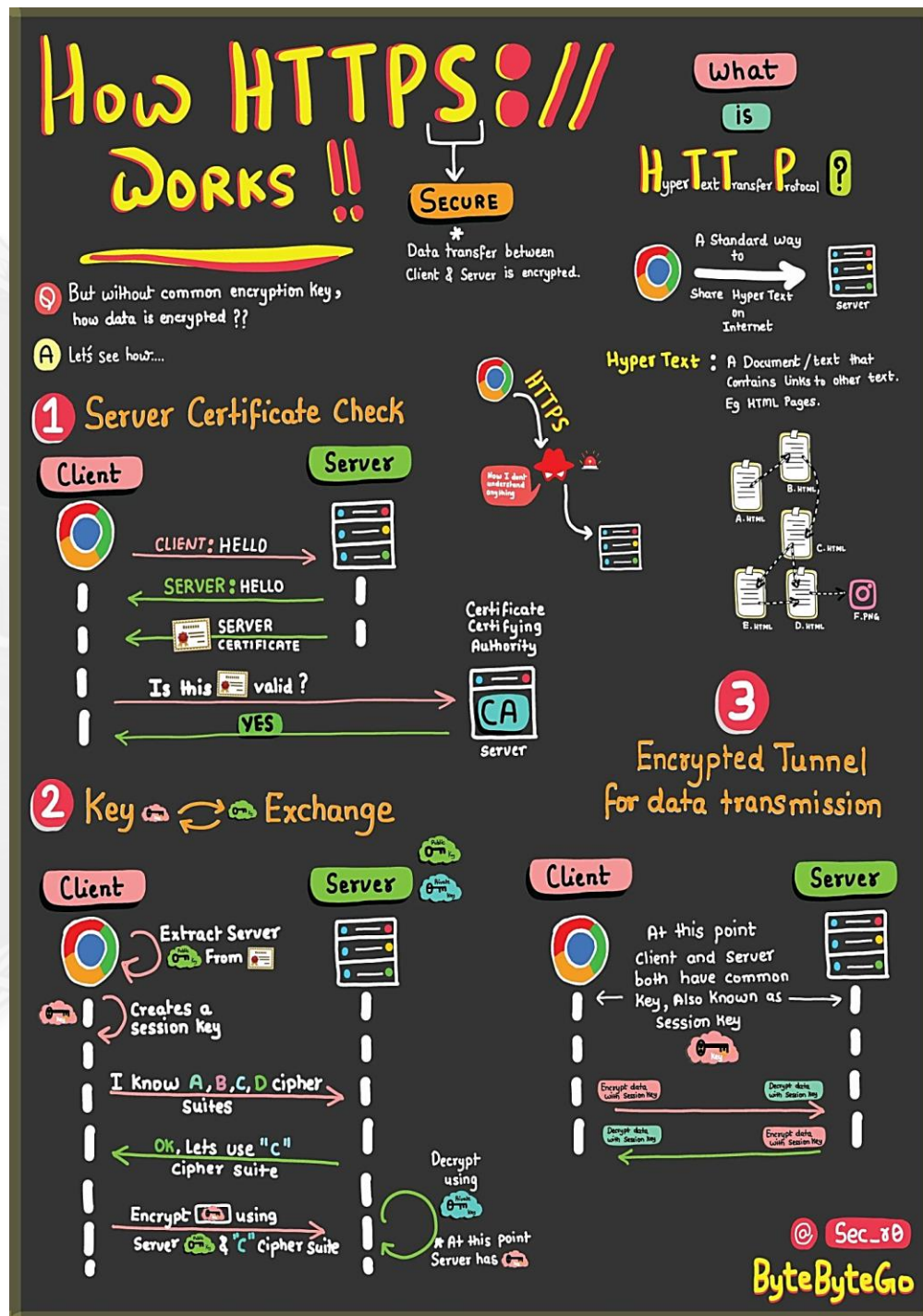


- **CADA** código de una biblioteca, framework, plugin, componente de terceros, etc. **ES UN RIESGO** incorporado en tu aplicación
 - Son ataques a tu cadena de suministro de software, "supply chain attacks"
 - Cualquier vulnerabilidad en un código de 3ºs que incluyas en tu app es ahora **TU vulnerabilidad**
 - Añade el requisito de **revisar periódicamente** las vulnerabilidades de **TODO** código de 3ºs (CVEs, **Tema 1**)
- Por suerte, existen herramientas que **automatizan** el proceso (SCA) (vistas luego)
 - Úsalas periódicamente en tus repositorios de código de aplicación (antiguos y nuevos)
 - Intégralas en tu pipeline CI/CD (**ASW**) y para el despliegue si se detectan vulnerabilidades
- Otra medida es **MINIMIZAR** el uso de componentes de terceros
 - Esto se llama **minimizar la superficie de ataque**
 - *¿Esto minimiza también el riesgo de verse afectado por vulnerabilidades zero-day?*

REQUISITOS HTTP



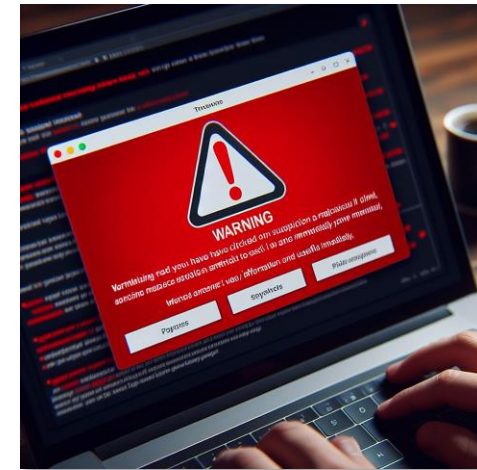
- Usa cabeceras de seguridad HTTP (véase Seminario 6)
- **SIEMPRE** usa HTTPS
 - Para evitar el **sniffing** de red y la intercepción de datos
 - Para evitar que el tráfico se modifique "en ruta"
 - *¿Qué puede pasar en este caso?*
- La configuración HTTPS debe ser robusta
 - Sigue las últimas normas, recomendaciones y prácticas
 - Disponibles en <https://ssl-config.mozilla.org/>



REQUISITOS DE SEGURIDAD SOBRE INFORMACIÓN A LOS USUARIOS

● Evita dar demasiada información a los usuarios, como...

- **Parámetros de URL:** Deben limitarse a datos irrelevantes (Ej.: idioma de la web)
 - *¿Por qué crees que existe esta recomendación? ¿Qué pasaría en los logs?*
- **Errores y log:** Los errores de usuario **NUNCA** pueden contener trazas de pila o errores de base de datos
 - Deben capturarse para que **no den información técnica al usuario**
 - Si tienes un XDR/SIEM (**Tema 3**), deben estar escritos en un formato compatible (y enviárselos a él)
 - ¡Tu XDR/SIEM también supervisará tus aplicaciones!
 - **Nunca debería guardarse información sensible** en los logs
 - *¿Por qué crees que esto podría ser un problema de seguridad?*
 - Si se detecta alguna **actividad sospechosa**, la aplicación debe hacerle **log** y enviar una alerta
 - Ej.: intentos de acceso a una función restringida, intentos excesivos de inicio de sesión, fallos en aplicaciones...
- **Elimina todos los comentarios** del front-end de la web; *¿Por qué?*



REQUISITOS DE FRAMEWORKS Y VERSIONES

● Características de seguridad de tus frameworks

- **USA SIEMPRE** las características de seguridad aplicables de tus frameworks
- En lugar de desarrollarlas tú (codificación, gestión de sesiones...); *¿Por qué?*
- Ej.: Accesos a la BBDD con su soporte para consultas parametrizadas
 - Ningún usuario ni aplicación debería poder interactuar directamente con la BBDD
 - Previene ataques de **inyección SQL** (vistos luego)

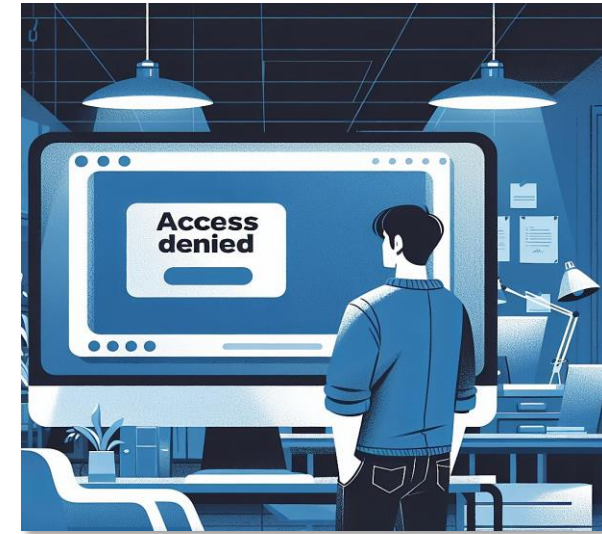
● Prepara los requisitos para enfrentarte a posible "deuda técnica"

- Hardcoding, software obsoleto / inseguro / no mantenido o sin parchear...
- Puertas traseras introducidas "para más agilidad" ☹
- ...
- *¿Por qué?*



REQUISITOS DE SEGURIDAD DE AUTORIZACIÓN

- Qué puede (o no) hacer un usuario autenticado (**AuthN**, ¿quién eres?) en la aplicación (autorización, **AuthZ** ¿qué puedes hacer?)
- Los requisitos determinan los métodos adecuados
 - **RBAC (Role-Based Access Control)** (recomendado): La aplicación usa diferentes grupos o roles
 - Con los permisos mínimos para su trabajo (Ej.: Probador, desarrollador, administrador de la base de datos...)
 - A los usuarios se les asigna uno o más roles
 - Cuando los usuarios se van o ascienden, sus roles se eliminan o cambian
 - **DAC (Discretionary Access Control)**: El propietario del recurso puede conceder o eliminar permisos libremente
 - **MAC (Mandatory Access Control)**: Esquemas de concesión de permisos descritos en el **Tema 3**
 - **PBAC (Permission-Based Access Control)**: Se conceden permisos a usuarios individuales o a varios usuarios a la vez



Nadie te impide usar varios métodos en distintas partes de una aplicación si es lo que necesitas

OTROS REQUISITOS DE SEGURIDAD IMPORTANTES



● Backups

- Todos los datos que maneja la página **deben estar en una copia de seguridad periódica**
- Además, incluye una copia de seguridad semanal **en otra ubicación geográfica**
- **También debe probarse y simularse el proceso de restauración de estas copias de seguridad**

● Principio de mínimo privilegio: Debe imponerse en todas las partes/niveles de la aplicación; Ej.:

- **Con accesos a bases de datos o APIs:** Crea dos cuentas de usuario
 - Una con permisos de **solo lectura** (Ej.: Para ejecutar solo sentencias **SELECT**)
 - Otra con permisos para implementar un **CRUD** de las entidades (**NUNCA** database owner (DBO))
 - La primera **debe usarse siempre en todas las operaciones posibles**
- **Implementaciones de APIs:** Se debe crear una cuenta de servicio **para cada API** (aislamiento)
 - Y cada API debe tener solo los permisos estrictamente necesarios para su funcionalidad
- **Repositorios de código fuente:** Concede permisos de lectura/escritura a los repositorios de código **ÚNICAMENTE a los miembros de los equipos de desarrollo adecuados**

¿CREES QUE LO HAS ENTENDIDO TODO? ¡AUTOEVALÚATE!

● Eres capaz de entender lo siguiente

- *Que todo componente de 3ºs incluido en una aplicación es un riesgo y que hay que minimizar su uso*
- *La importancia de configurar el protocolo HTTP lo más robustamente posible (cabeceras, cifrado...)*
- *Que hay **otra serie de requisitos de seguridad importantes** que debemos considerar, como*
 - *Hacer **backups** de todo el código y datos de la aplicación (periódicamente, simulando restauraciones)*
 - *Usar todas las funcionalidades de seguridad que tengamos a nuestro alcance (librerías, **frameworks**)*
 - *Que la situación no va a ser la ideal debido al concepto de deuda técnica, y que tenemos que considerar su existencia*
 - *Que debemos aspirar al menos a usar roles para gestionar usuarios como requisito*
 - *La importancia de las **parametrized queries** a la hora de hacer peticiones a los SGBD*
 - *Que los parámetros de la URL son un punto de entrada para ataques frecuente*
 - *Y que jamás debemos otorgar a un usuario y/o parte de la aplicación más privilegios que los estrictamente necesarios*



[< Ir al Índice](#)

[Threat modeling >](#)



DISEÑO SEGURO

Aspectos de seguridad en el diseño de aplicaciones



Departamento de Informática
Universidad de Oviedo



Universidad de Oviedo

¿QUÉ TE VAS A ENCONTRARTE EN ESTE BLOQUE?



● En este bloque aprenderás

- Los conceptos principales que modelan un **diseño seguro**
 - Viéndolos como una forma de plasmar los requisitos en elementos concretos de actuación
- Qué es y cómo se hace el **threat modeling**
 - Incluyendo los **actores** involucrados y la importancia de lo que se obtiene al final del proceso

FALLO DE DISEÑO VS BUGS

- Todos los requisitos de seguridad vistos deben ponerse en práctica en el diseño
 - Para así evitar en lo posible los fallos de diseño
- Los fallos de diseño **NO** son bugs de seguridad
 - Un **fallo de diseño** permite a los usuarios hacer cosas que no deberían
 - Un **bug** es un fallo en el código de la implementación de una característica
- **Recuerda:** Cuanto 1º se descubra un fallo de diseño relativo a seguridad, más barato es arreglarlo
- Para hacerlo, necesitamos
 - Seguir una serie de **principios generales de diseño de software** al diseñar los requisitos (siguiente hoja)
 - Implementar procesos de **threat modeling** (siguiente subsección)



- **Protección de datos sensibles:** Implementar todo lo dicho en requisitos sobre protección de datos
 - Ej.: Decidir cuánto tiempo se guardan datos sensibles y borrarlos automáticamente una vez transcurrido
 - Siguiendo la legislación vigente (**ASLEPI**)
 - Planifica posibles brechas de seguridad: <https://www.incibe.es/emprendimiento/publicaciones/guias-y-estudios/guias/guia-de-como-manejar-una-brecha-de-seguridad>
- **Separación de código/funcionalidad por diseño:** Separar el código que lidia con aspectos de seguridad del código funcional
 - Clases separadas, otra aplicación, usar autenticación de un proveedor externo tipo Google, etc.
 - Incluso en servidores o contenedores separados (por un firewall)
 - También con los logs y monitorización, IDS/IPS, WAFs... (ver **Tema 5**)
 - También **separar el código de administración del “estándar”**
- **Diseño de la gestión de secretos de la aplicación**
 - **Definir dónde guardamos los secretos** (conexión a la BBDD, certificados, API Keys,...)
 - Definir una **secret store**, incorporarla a la cadena CI/CD o definir cómo acceder por programa



CONCEPTOS DE DISEÑO SEGURO

● Al diseñar software, recuerda siempre estos principios

- **Never Trust, Always Verify / Zero trust / Assume breach:** Solo usar datos previamente validados en el servidor para tomar decisiones
 - **Asumir que cualquier dato podría estar comprometido** al diseñar la aplicación
- **Denegar por defecto:** Diseñar todas las funciones para que comprueben siempre si un usuario está autorizado a usarlas
 - **Verificar siempre la identidad (AuthN) antes de comprobar si se tiene permiso** para hacer algo (**AuthZ**)
 - No importa dónde esté la funcionalidad a la que accedas (**backend**, terceros, API...)
 - En ambos sentidos (llamadas desde la aplicación y hacia la aplicación)
 - **Recomprueba** si sigues teniendo acceso a cada página o función de la app, aunque solo se recargue
 - **Cierra siempre correctamente** una transacción en caso de fallo
 - Bloquea **cualquier protocolo, puerto, verbo o método HTTP** que la aplicación no use
- **Aislamiento por diseño:** Diseñar el sistema para desplegar **solo una aplicación por servidor**, PaaS o contenedor
 - Servidor, PaaS o contenedor (**ASR**, L-62 “Princesa de Asturias”)



CONCEPTOS DE DISEÑO SEGURO

- **CSRF:** Identifica los puntos en los que el usuario hace operaciones sensibles
 - Compras, cambios de clave...
 - Para pedirle su clave, un captcha o un token que solo él conozca *¿Por qué?*
- **NUNCA uses datos de producción para testing:** Diseña fuentes de datos separadas
- **Diseña cómo proteger el código fuente de la aplicación:** Los supply chain attacks son críticos; necesitas una estrategia para proteger tus repositorios
 - **Políticas y permisos de acceso a los repositorios,** ofuscación, backup, monitorización y log...
 - Ser open source permite a cualquiera ver el código y sugerir cambios
 - Pero los cambios deben supervisarse adecuadamente para evitar introducir código malicioso
 - Puedes seguir el OWASP Software Component Verification Standard
 - <https://owasp.org/www-project-software-component-verification-standard/>





Threat modeling

Identificando amenazas potenciales antes de construir la aplicación



THREAT MODELING (“EVIL BRAINSTORMING”)

- **Identificación de potenciales amenazas** para un negocio, sistema o software
- **Brainstorming** donde se piensan, buscan y definen los posibles peligros a los que nos podemos enfrentar. Ejs.:
 - Intercepción o filtrado de datos con su posterior venta, y el impacto que puede tener
 - Cómo protegerse ante posibles ataques que identifiquemos
 - ... (cualquier peligro, cualquier problema que imagines, piensa mal, piensa en peligros, adopta el lado oscuro como si fuera tu padre)
- **Si identificamos alguno debemos...**
 - **Cambiar el diseño** de la aplicación para contrarrestarlo
 - Que un miembro del equipo responsable de ello **acepte el riesgo** de una forma documentada (**DPPI**)
- **Es algo que puede ser muy complejo, aquí daremos una introducción**
 - Para identificar riesgos nos podemos apoyar en los elementos del MITRE ATT&CK (**Tema 6**)



THREAT MODELING (“EVIL BRAINSTORMING”): ¿QUÉ NECESITAMOS?

● Un representante de cada stakeholder (parte interesada) de la aplicación

- El cliente
- Un **especialista en seguridad ofensiva**
- **Arquitecto del sistema**
- Responsables de la **plataforma de despliegue**
- ...

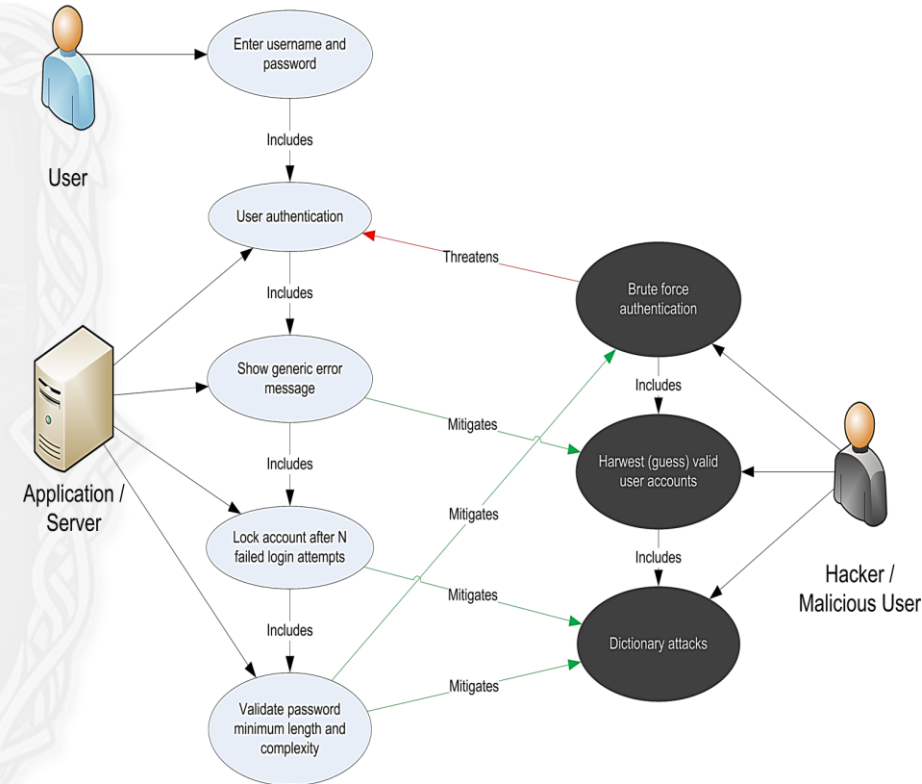
● Cada grupo debe aportar los riesgos que ve en el sistema

- *“Si fueras a atacar el sistema, ¿Cómo lo harías?”*
- *“¿Quiénes crees que atacarían el sistema?”*
- *“¿Cómo te prepararías para un ataque?”*
- *“¿Qué podría ir mal?”*
- *“¿Cómo protegemos al usuario?”*
- *“¿Qué es lo peor que puede pasar?”*
- ...



THREAT MODELING (“EVIL BRAINSTORMING”)

- Deben verlo desde la mentalidad “del mal”, como si fueran adversarios
 - Convertir las “user stories” en “abuse stories” o “casos de uso negativos”
 - Ej.: Si una aplicación se supone que hace X, ¿qué pasa si hace Y?
 - Requiere práctica y detalle
- Pero existen metodologías que facilitan su implementación
 - Attack trees: <https://www.mytechiebits.com/AttackTrees>
 - STRIDE: <https://www.koenig-solutions.com/blog/stride-methodology-in-threat-modelling>
 - PASTA: https://owasp.org/www-pdf-archive/AppSecEU2012_PASTA.pdf
- Permiten al equipo hacerse preguntas necesarias
 - Para cubrir todo lo que puede ir mal



Un "caso de uso negativo" ¿Quieres saber cómo empezar a hacer esto en tu software? OWASP te ayuda aquí:

https://owasp.org/www-community/Threat_Modeling_Process

THREAT MODELING (“EVIL BRAINSTORMING”)

- El **threat modeling** permite pensar en...
 - Qué tiene **nuestra aplicación de valor** (dinero, datos...)
 - **Cómo** alguien podría robarlo, manipularlo (reputación, DoS...), romperlo o usarlo mal
- Las respuestas son una “**lista de problemas**”
 - Hay que evaluar cuáles **son riesgos**, asignándoles una **puntuación (DPPI)**
 - Y elegir si **mitigarlos** (cambiar los requisitos, el diseño, el código...) o **aceptarlos**
 - Solo por la dirección del proyecto
 - Todo este proceso debe quedar **documentado**
 - El método cambia según cómo se hace el diseño
 - En **cascada** se hace una sesión al comienzo
 - Un **diseño iterativo** funciona mejor con varias sesiones más cortas en cada iteración...

Non-invasive attack	Remote	Local	Potential damage	Probability of attack	Mitigation
Communication protocol	✓		Access to all data on network; escalation of privilege on device	High	Secure communication protocol (TLS) with a True Random Number Generator
Authentication	✓	✓	Impersonation of device on network, access to all assets	High	Follow proper authentication guidelines, fuzz testing
Code vulnerabilities	✓	✓	Control of device	High	Isolate application memory in CPU
JTAG (debug) access		✓	Full control of device; access to all assets	High	Disable JTAG; re-enable only with secure protocol
GPIO control		✓	Change device state	Medium	Monitor GPIO for unexpected behavior, fuzz testing
Side channel (DPA/TA)		✓	Access to keys	Medium	Side channel resistant cryptographic algorithm implementations
External bus monitoring		✓	Access to keys and application data	Low	Encrypt external bus interfaces

Fuente: <https://www.synopsys.com/designware-ip/technical-bulletin/using-threat-models-2017q4.html>

¿CREES QUE LO HAS ENTENDIDO TODO? ¡AUTOEVALÚATE!



● Eres capaz de entender lo siguiente

- *Que el diseño debe ser un fiel reflejo de los requisitos*
- *Que, a la hora de proteger los datos, siempre debe seguirse una política existente aplicable en el contexto de la aplicación*
- *Por qué es mejor diseñar asumiendo que cualquier dato puede ser comprometido*
- *Que, ante la duda sobre lo que hacer, lo más seguro es denegar una petición*
 - *Incluyendo que antes de autorizar algo primero hay que asegurarse verificar la identidad y luego los permisos sobre ese algo de dicha identidad*
- *Que es mejor diseñar una aplicación por partes aisladas todo lo posible*
 - *Usando distintos criterios, incluyendo las fases en las que se divide el desarrollo (datos de testing distintos de datos de producción)*
 - *Y, además, proteger el acceso al código fuente en aquellos casos donde no sea público*
- *Que en la fase de diseño debe eliminarse toda posibilidad de "hardcodear" información*
- *Qué es, quien realiza y para qué sirve el threat modeling*
 - *Incluyendo entender que, al final del proceso, tendremos una lista de riesgos identificados que debemos estudiar cómo tratar*

[< Ir al Índice](#)

[AuthN & AuthZ >](#)

[Log y errores >](#)



12 34 CREACIÓN DE CÓDIGO SEGURO

Aspectos a tener en cuenta en la implementación



Departamento de Informática
Universidad de Oviedo



Universidad de Oviedo

¿QUÉ TE VAS A ENCONTRARTE EN ESTE BLOQUE?

● En este bloque aprenderás

- Criterios para **seleccionar los frameworks** y lenguajes de desarrollo
- Un **algoritmo** concreto para **validar datos**
- Elementos importantes para una **implementación de una web segura**
- Criterios para implementar una **gestión de identidades y autenticación** seguras
 - Incluyendo la importancia de **externalizar el servicio** y la mejor política de autorización
- Criterios a la hora de **implementar el log** de una aplicación
 - Incluyendo su **protección**



SELECCIONAR EL FRAMEWORK Y EL LENGUAJE DE PROGRAMACIÓN

● Para elegir frameworks / lenguajes / dependencias

- Elige los compatibles con tu entorno de desarrollo
- Elige **versiones LTS** (Long-Term Support), nunca las inestables
- Mira el **historial de vulnerabilidades** en un repositorio de **CVE** (**Tema 1**)
- **No uses** “el ultimo framework o lenguaje de moda” *¿Por qué?*
- **Minimiza** la cantidad de librerías, componentes, frameworks, etc. usados (**superficie de ataque**)
- Escanea todo periódicamente con una herramienta **SCA** (visto luego)

● No uses lenguajes no memory-safe como C/C++

- Introducen problemas de seguridad adicionales
- Una alternativa memory-safe es Rust



DATOS NO CONFIABLES: VALIDACIÓN

- Esto es **obligatorio**
- **Siempre validar del lado del servidor**
 - Hagamos o no validaciones en el cliente (Ej.: con JavaScript)
 - Bueno para **usabilidad**
 - Inútil para seguridad
 - **Antes** de usar cualquier entrada de usuario
- Sigue este esquema como guía



VERBOS HTTP

● Si una aplicación usa el protocolo HTTP...

- Hay varias acciones que puede realizar y que se modelan a través de **verbos** HTTP (**GET**, **POST**, **PUT**, **DELETE**...ver **SEW**)
- **Elimina el soporte de todo verbo no usado; ¿Por qué?**
- Una aplicación web típica normalmente sólo usa **GET**, **POST**, **OPTIONS** y **HEAD**

● Se recomienda seguir los CIS Benchmarks para desactivarlos

- Podemos hacerlo a nivel servidor web o framework *¿Qué ventajas / desventajas tienen ambas formas?*
- ¡Acuérdate del **Tema 3**!

HTTP Method	Request has body	Response has body	Safe	Idempotent	Cachable
GET	NO	YES	YES	YES	YES
POST	YES	YES	NO	NO	YES
PUT	YES	YES	NO	YES	NO
DELETE	YES	YES	NO	YES	NO
TRACE	NO	YES	YES	YES	NO
OPTIONS	NO	YES	YES	YES	NO
CONNECT	NO	YES	NO	NO	NO
PATCH	YES	YES	NO	NO	NO

Ten cuidado, especialmente con las aplicaciones REST. Fuente:

<https://intmain.co/http-request-methods-rest-api-verbs>

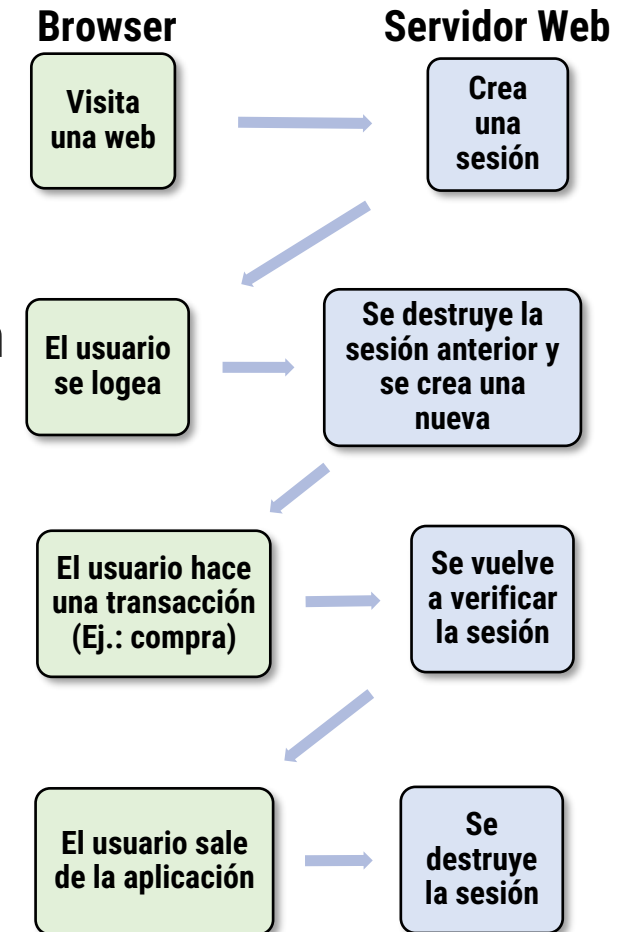
MANEJO DE SESIONES

● Una correcta gestión de IDs de sesión requiere

- Deben tener **al menos 128 caracteres**
- Deben ser **impredecibles** (aleatorios) para evitar su **precalculación**
 - Se debe usar un generador de IDs aleatorios reconocido y no propio
 - **Usar, si existe, el manejador de sesiones del framework**
- El usuario debe recibir un nuevo ID **cada vez que se autentique**
- Debe tener una **fecha de expiración**, aunque el usuario siga en la aplicación
- Solo debe **transmitirse por canales cifrados**
- **Nunca aceptar** un ID de sesión que no se haya generado por la aplicación
 - Debe generar una **alerta de log/SIEM** y bloquear la IP (incidente de seguridad)
- Deben **regenerarse** en cada autenticación, re-autenticación
 - U otro evento que cambie el nivel de privilegio del usuario

● Muchos frameworks ya tienen esto integrado

- Investígalo y **no lo programes tú**





Identidad y autenticación (AUTHN)

Cómo implementar la comprobación de identidad y los permisos
de nuestros usuarios

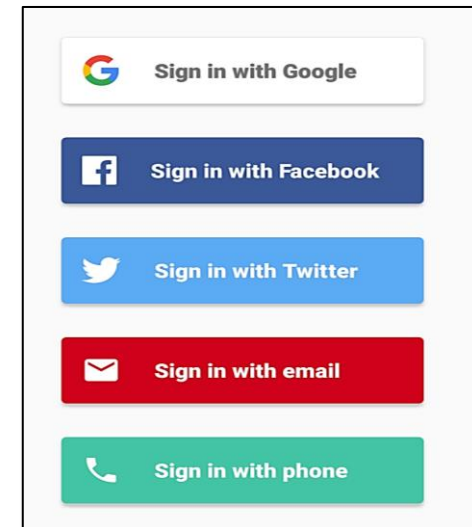
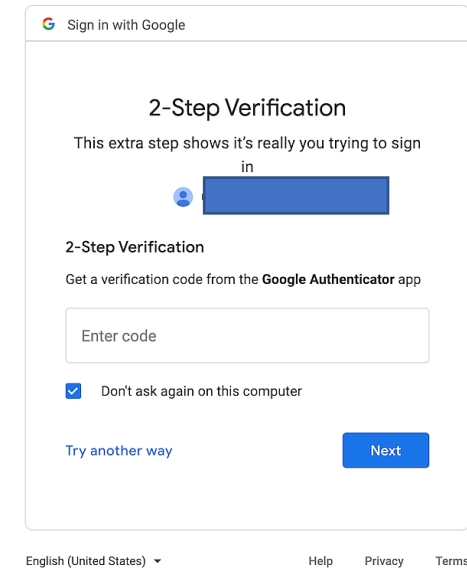


EVITA CREAR TU PROPIO SISTEMA DE AUTENTICACIÓN



SSI

- Si existe un mecanismo de autenticación para todos (o la mayoría) de los usuarios, úsalo; *¿Qué ventajas tiene esto?* Ej.:
 - Sistema de identidad basado en Active Directory en dominios Windows ([ASR](#))
 - Los proveedores de nube tienen sus propios sistemas de autenticación. P ej.:
 - **Google:** <https://cloud.google.com/docs/authentication>
 - **Amazon:** <https://aws.amazon.com/es/cognito/>
 - LinkedIn, Twitter, Facebook...
 - ... (o varias, para evitar el "vendor locking")
- Usa uno que sea compatible con nuestros requisitos y contexto
 - Esto facilita el Single Sign-On (siguiente diapositiva)
 - Pero evalúa sus riesgos primero; *¿Cuáles son?*
- Si esto no es una opción, porque tienes requisitos particulares...
 - Utiliza un protocolo establecido, como OAUTH: <https://oauth.net/getting-started/>

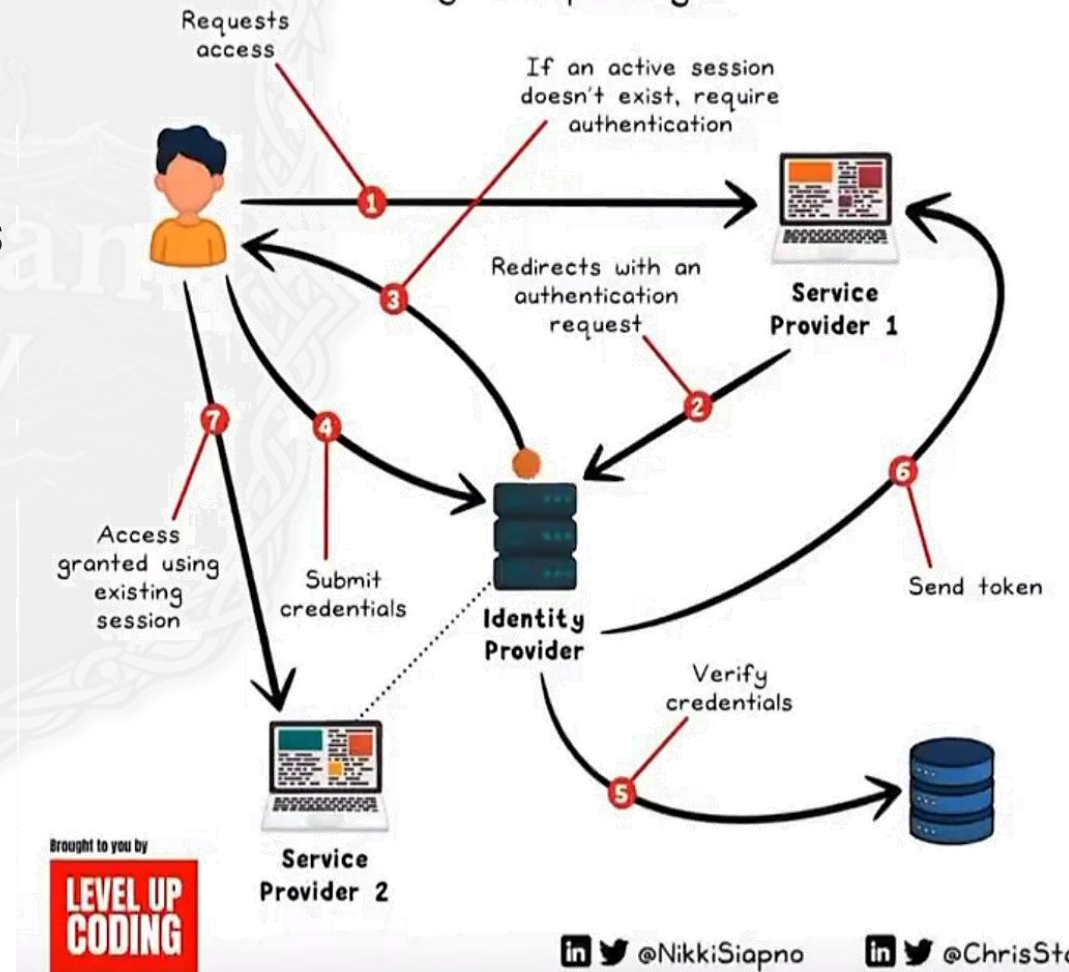


USAR SIEMPRE UN PROVEEDOR DE AUTENTICACIÓN DE TERCEROS

- Usar un proveedor de terceros también facilita el uso de Single Sign-On (SSO)
 - Al hacerlo, **tienes acceso a una o varias aplicaciones** vinculadas a dicho servicio de autenticación
 - *¿Puedes recordar algún ejemplo?*
- Facilita la administración de cuentas; *¿Por qué?*
 - *¿Mejora la seguridad también?*
- Mejora la experiencia del usuario; *¿Por qué?*

Single Sign-On (SSO)

by levelupcoding.co





Manejo de errores, log y monitorización

Qué hacer con la información que podemos obtener de nuestra aplicación mientras está en uso



GESTIÓN DE ERRORES, LOG Y MONITORIZACIÓN: REGLAS DE MANEJO



- **TODOS** los errores de aplicación deben capturarse y gestionarse

- Hacer un **catch** global (en el **main**) es una buena idea como último recurso
- *¿Tiene sentido (o es conveniente) dejar ese catch vacío?*

- Errores de usuario y de aplicación se gestionan diferente

- Usuarios "estándar" **NUNCA** ven información técnica o detallada de errores
- Errores de seguridad (login, control de acceso, validación...) **generan alertas**
 - Enviados a un IDS/IPS o a un XDR/SIEMs que los analicen (**Temas 3, 5**)
 - Si el error incluye información de otros sistemas...
 - **Codificar** el contenido, **eliminar todos los caracteres no imprimibles** y **limitar el tamaño** de la información registrada; *¿Por qué?*

- Normas sobre la información escrita en los logs

- **Nunca escribas información privada o personal identificable**
- **Usa un formato estandarizado** (¡el de XDR/SIEM!); Ej.:
 - **Tipo** de evento (seguridad/otro); **Cuándo** (marca temporal); **Dónde** (sistema, IP, URL, subdominio...); **Resultado** del evento; Información **adicional**

```
192.168.1.4 - - [26/Apr/2018:17:18:54 +0200] "GET /2017/05/30/how-to-retrieve-ii
s-http-logs-remotely/ HTTP/1.1" 301 671 "-" "Mozilla/5.0 (Windows NT 6.1; WOW64;
Trident/7.0; rv:11.0) like Gecko"
```

¿Cuál es el mejor?

A screenshot of a Twitter error message. It features the Twitter bird logo at the top. Below it, the word "Error" is centered. The message reads: "Something went wrong, but don't fret — it's not your fault. Let's try again." At the bottom, there are two buttons: a blue "Refresh" button and a white "Log out" button with a blue outline.

A screenshot of a Proton login form. The title is "Iniciar sesión". Below it, a subtitle says "Introduce los detalles de tu cuenta de Proton." The form has two input fields: "Dirección de correo electrónico o nombre de usuario" with the text "meloinvento" and "Contraseña" with masked characters. Below the password field is a red error message: "Este nombre de usuario no existe. Intentalo de nuevo con un nombre de usuario diferente." At the bottom, there is a checkbox for "Mantener mi sesión iniciada" with the text "Recomendado en dispositivos de confianza. ¿Por qué?" and a blue "Iniciar sesión" button.

GESTIÓN DE ERRORES, LOG Y MONITORIZACIÓN: REGLAS DE MANEJO

- **No solo hacer log de errores, sino de cualquier evento significativo**
 - Esto se establece en los requisitos / diseño
- **Los logs son extremadamente importantes, deben protegerse**
 - Evita accesos no autorizados
 - Inclúyelos como parte de tu estrategia de **backup**
 - Registra y monitoriza el acceso a los mismos
 - Guárdalos cifrados y en un servidor aislado y seguro (**Tema 5**)
 - Asegúrate de que sean accesibles para **equipos de respuesta a incidentes**
 - Enlazando con la asignatura optativa de Informática Forense y Auditoría (**IFA**)
 - Bórralos periódicamente
 - Crea una política para ello que siga las normativas legales de tu país



¿CREEES QUE LO HAS ENTENDIDO TODO? ¡AUTOEVALÚATE!



● Eres capaz de entender lo siguiente

- *Los criterios para elegir un framework o lenguaje de desarrollo correctamente*
- *El algoritmo con el que debes implementar la validación de todo dato de entrada*
- *Que debes minimizar el nº de verbos HTTP que tu aplicación admite*
- *El algoritmo a implementar cuando se deban gestionar sesiones*
- **Sobre identidad y autenticación**
 - *Por qué no se debe crear “nunca” nuestro propio sistema para autenticar usuarios*
 - *El concepto de Single Sign-On*
 - *Los distintos tipos de autorización que puedes implementar, sus ventajas y desventajas*
- **Sobre log**
 - *Qué cosas no deben aparecer nunca en un log de tu aplicación*
 - *Que tu aplicación seguramente esté en un ecosistema más grande, y que el formato de los logs debe adaptarse al mismo*
 - *Que la manipulación y alteración de los logs por parte de personas no autorizadas nos deja ciegos ante ataques que se hayan producido*

[< Ir al Índice](#)



AST (APPLICATION SECURITY TESTING)

Herramientas y técnicas para comprobar la seguridad del software



[Estáticas >](#)

[Dinámica >](#)



Departamento de Informática
Universidad de Oviedo



Universidad de Oviedo

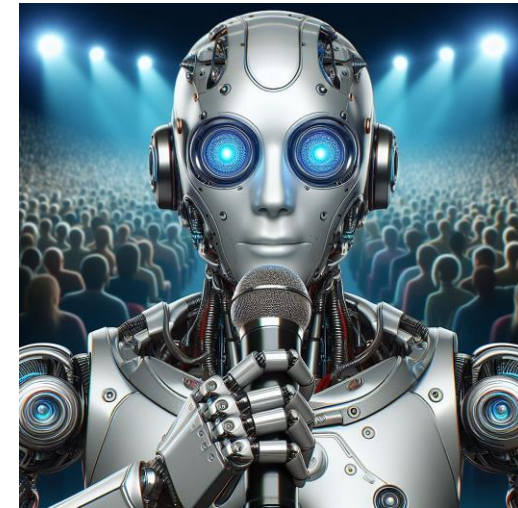
¿QUÉ TE VAS A ENCONTRAR EN ESTE BLOQUE?

● En este bloque aprenderás

- Qué son las herramientas **SCA**
- Qué son las herramientas **SAST**
 - Incluyendo cómo tener ambas fácilmente si usas **GitHub**
- Qué son las herramientas **DAST**
- La importancia de la integración de todas estas herramientas en un ciclo de desarrollo **SecDevOps**



- **Las actividades de testing detectan problemas de seguridad**
 - Mismo método para hacer pruebas funcionales (**CVVS**), pero para los “abuse stories” o “*casos de uso negativos*”
- **La seguridad de nuestro software podría evaluarse por 3ºs**
 - Normalmente, para ser aceptado en varios países / contextos
 - Se usan los Common Criteria
 - <https://www.commoncriteriaportal.org/index.cfm>
 - Cada país tiene un organismo certificador; el CCN es el de España
- **El testing implica introducir herramientas AST en el proceso de desarrollo del software**
 - Especialmente si seguimos un modelo DevOps con un esquema CI/CD (**ASW**)
 - Todas estas herramientas deben integrarse en nuestro pipeline
 - Lo convertiremos en SecDevOps
 - ¿*Cuáles?* Depende de nuestras herramientas de desarrollo / entorno (nube...)





Herramientas estáticas: SCA y SAST

Analizando el código antes de la ejecución



HERRAMIENTAS SCA (SOFTWARE COMPOSITION ANALYSIS)



- **Escanean automáticamente el código open source / artefactos (contenedores)**
 - **Inventario:** Recopilan todos los componentes open source, dependencias directas e indirectas
 - *¿Por qué es importante esto ?*
 - **Cumplimiento de licencias:** Todas deben ser compatibles con la política de la organización
 - **Vulnerabilidades de seguridad:** identifica **vulnerabilidades conocidas** en estos componentes (**CVE**)
 - **Es su principal función** (ver transparencia siguiente)
 - Informan si la aplicación **llama a código vulnerable**, sugieren mitigaciones, gestionan actualizaciones/parches
 - **Características avanzadas:** Obligar a todo componente open source a cumplir políticas de la organización
 - Con respuestas automatizadas (solicitar aprobación del componente, impedir la construcción...)
 - Avisan de las vulnerabilidades de un componente antes de hacerle un **pull request** y que entre al sistema
- **Ejemplos**
 - <https://sourceforge.net/software/software-composition-analysis-sca/>
 - <https://www.paloaltonetworks.com/cyberpedia/what-is-sca>

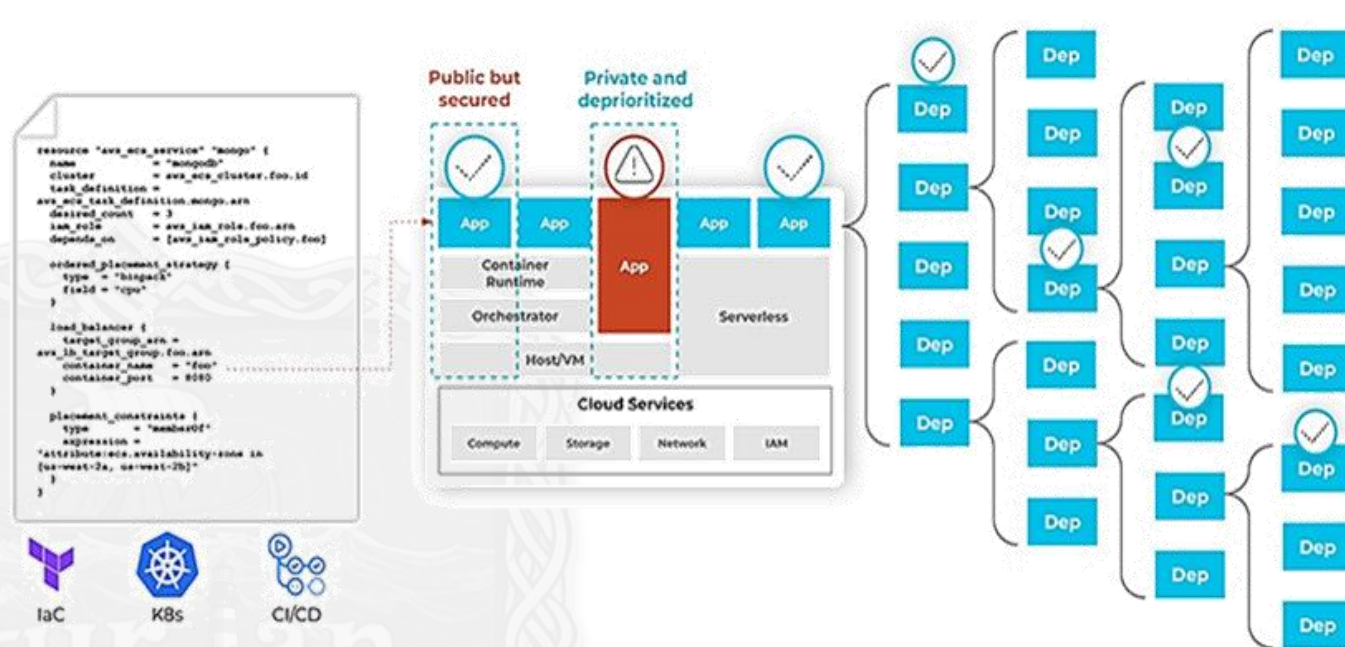
HERRAMIENTAS SCA



- Las SCA analizan el código de esas dependencias open source

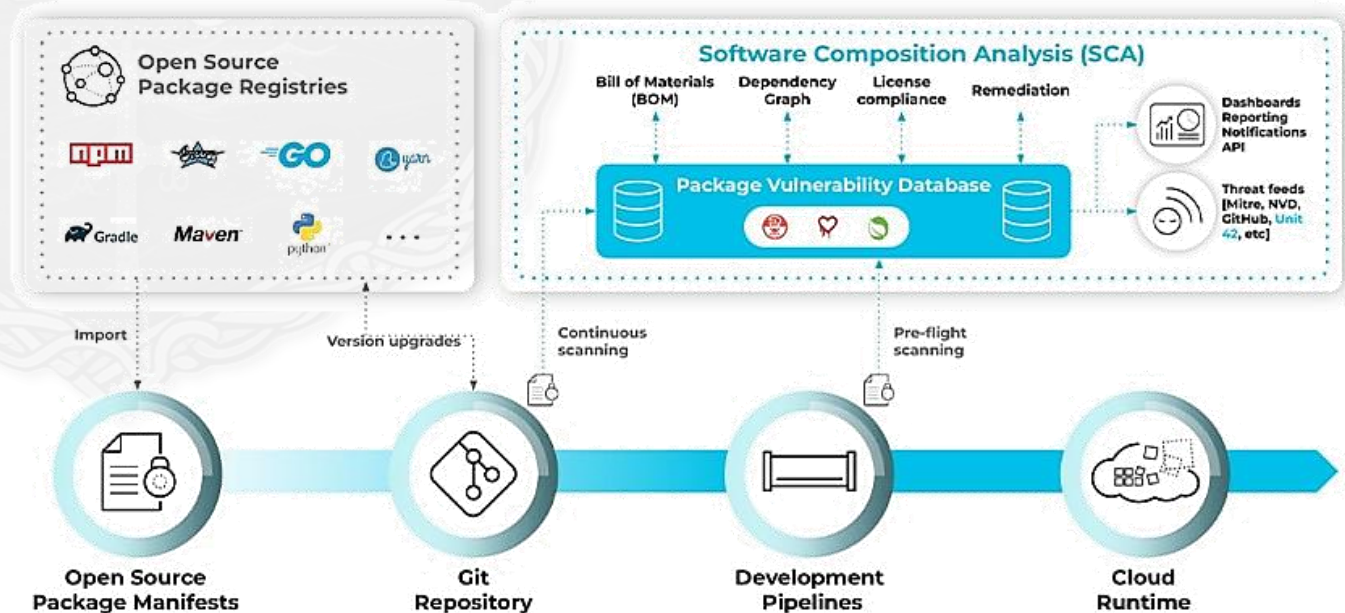
- Evitan que se introduzcan sus vulnerabilidades en nuestra aplicación
- Si no usamos una SCA, alguna dependencia vulnerable “envenenaría” nuestro software 😞

- Todo el proceso de análisis se puede integrar automáticamente en el pipeline de desarrollo



Una herramienta SCA analiza el código de todas las dependencias de código abierto de la aplicación dentro de la cadena de aplicación. Fuente:

<https://www.paloaltonetworks.com/cyberpedia/what-is-sca>



HERRAMIENTAS SAST (STATIC APPLICATION SECURITY TESTING)



● Prueban estructuras internas o código de la aplicación

- No su funcionalidad (**pruebas de caja blanca**)
- **Ventajas**
 - Soportan muchos lenguajes: PHP, Java y .Net, C, C++, C#...
 - Descubren vulnerabilidades complejas durante las 1^{as} etapas de desarrollo, que se pueden resolver rápidamente
 - Detallan el problema, incluida la línea de código
 - Se puede integrar en un entorno SDLC existente
- **Inconvenientes**
 - Escalar esta tecnología a grandes desarrollos puede ser complejo
 - Suelen ser **inexactas**: muchos falsos positivos y falsos negativos
 - Aún peor con lenguajes de tipado dinámico
 - No pueden probar la aplicación en el entorno real
 - No detectan vulnerabilidades en la lógica de la aplicación
 - 0 configuraciones inseguras

Fuente: <https://www.mend.io/resources/blog/sast-vs-sca/>

WhiteSource	
SAST VS SCA	
DON'T GET IT TWISTED	
DETECTS VULNERABILITIES IN PROPRIETARY CODE Detects potential vulnerabilities, only in code written in house. Not realistically applicable on open source.	DETECTS VULNERABILITIES IN OPEN SOURCE Detects open source components with known vulnerabilities. Detailed security information for each vulnerability is publicly available.
REQUIRES SOURCE CODE ACCESS Only analyzes source files, which means it scans your source code.	ACCESS TO SOURCE CODE NOT REQUIRED Identifies both source files and binaries. It doesn't scan your source code, but only calculates digital signatures for all libraries.
COMPLICATED REMEDIATION PROCESSES Each remediation process starts with an investigation to validate the vulnerability. Once confirmed, developers need to define the best remediation path.	EASIER TO FIX VULNERABILITIES As 97% of all open source vulnerabilities have a fix, developers simply need to patch or download the latest version.
LIMITED SDLC INTEGRATION The key is to detect issues as early as possible. Currently integrates with CI servers and IDEs.	END-TO-END SDLC COVERAGE Open source security requires shifting security left and right. Integrates with IDEs and repos all the way to post-deployment for vulnerabilities discovered years after release.
HIGH FALSE POSITIVES Scans for patterns of security issues, therefore results in high false positives anywhere between 30% to 70%.	NO FALSE POSITIVES For vendors associating vulnerabilities to components with pinpoint accuracy, in an accurate way, there will be zero false positives.
TIME CONSUMING Scans can take up to a few hours, and then requires additional overhead for research and validation.	NO IMPACT ON BUILD OR REPO Runs within seconds with no impact on build, no matter what your project size.
COVERS A SPECIFIC SECURITY ASPECT Only covers AppSec issues in 10% to 20% of your codebase.	COVERS ALL OPEN SOURCE RISKS AND INCREASES AGILITY Covers all aspects of open source usage management including security and compliance, using automated workflows to simplify developers everyday tasks.

HERRAMIENTAS SAST

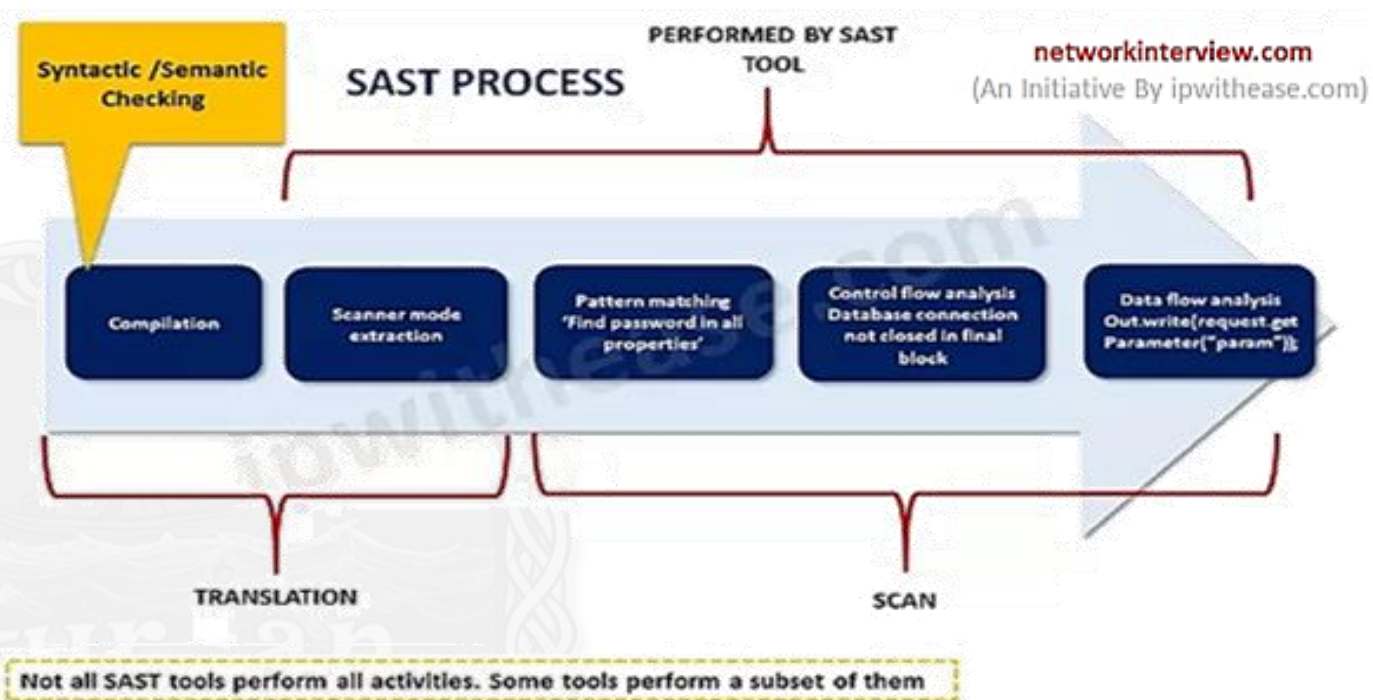


- Un SAST es como una extensión del proceso de compilación del software (**DLP**)

- Pero en vez de errores de tipo, busca patrones conocidos de malas prácticas de código
- Es como un procesador de lenguaje especializado en “pifias”

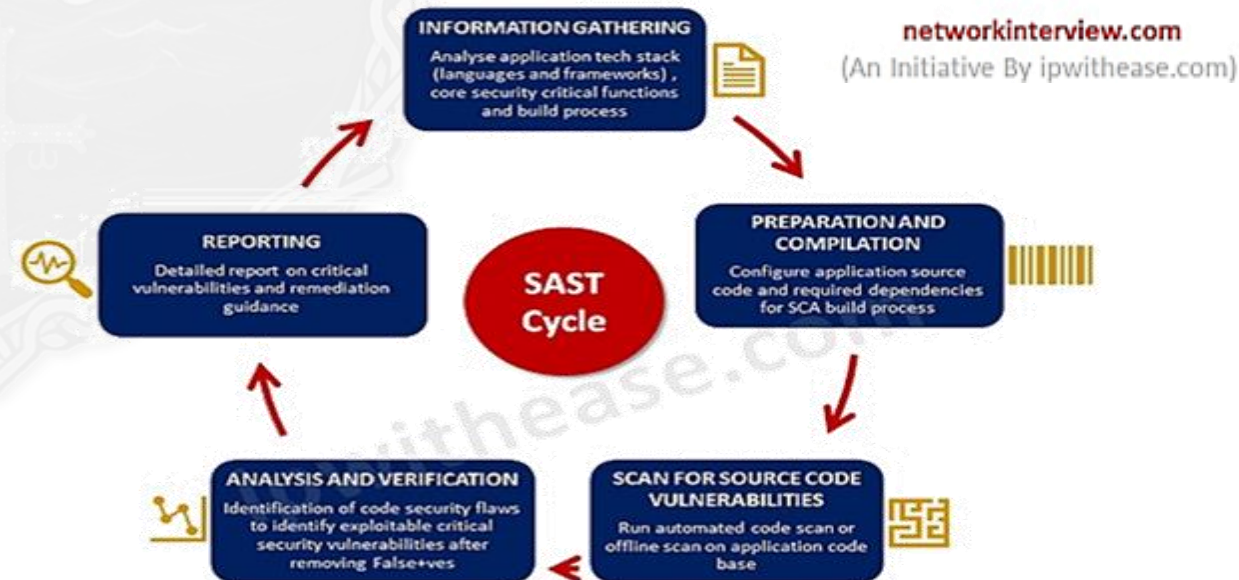
- Tras su análisis, da un informe

- **Independiente** de la salida del compilador
- Con los detalles de los problemas encontrados y soluciones sugeridas



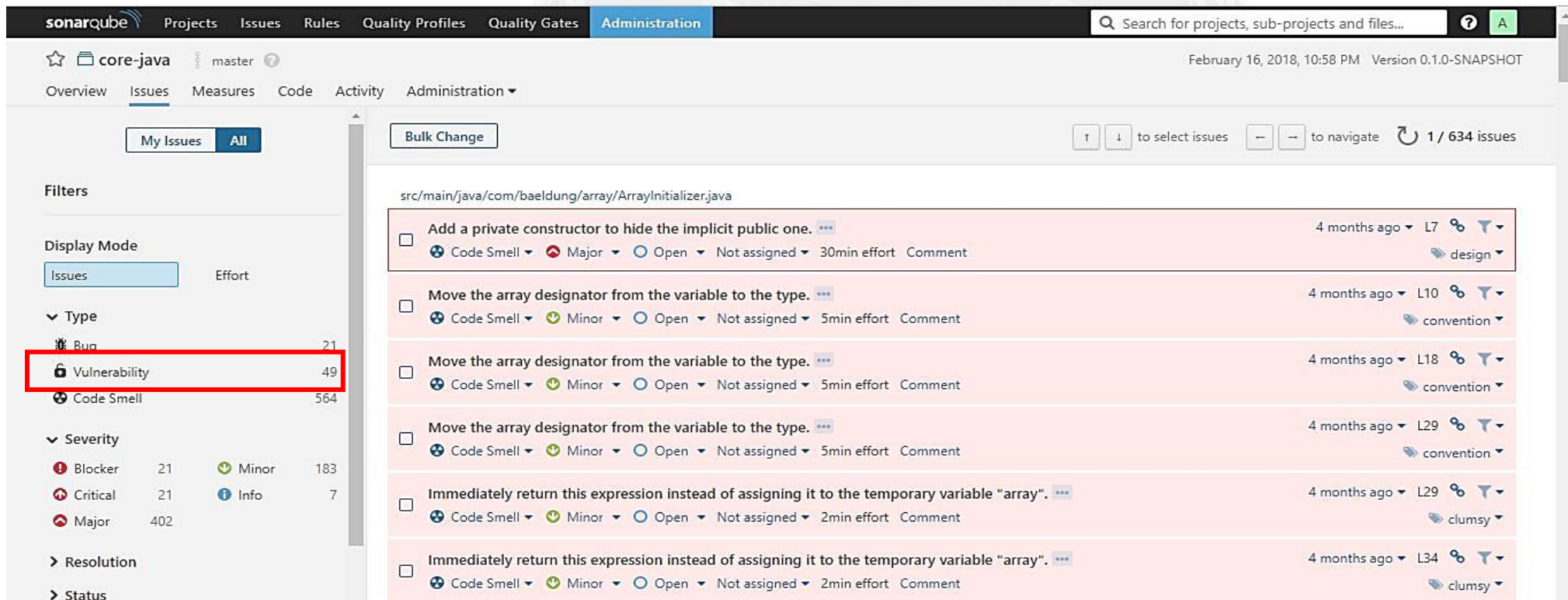
Una herramienta SAST actúa tras el análisis sintáctico y semántico (**DLP**).

Fuente: <https://networkinterview.com/sast-static-application-security-testing/>



HERRAMIENTAS SAST

- Una herramienta SAST muy popular es SonarQube (vista en labs)
 - Más herramientas: <https://www.g2.com/categories/static-application-security-testing-sast>
 - Pasar una SAST y una SCA a vuestro TFG es un buen plus (guiño, guiño 😊)
 - GitHub también tiene soporte automatizado e integrado para SAST e SCA



The screenshot shows the SonarQube web interface for the project 'core-java'. The 'Issues' tab is selected, and the 'Vulnerability' filter is highlighted in the left sidebar. The main panel displays a list of issues for the file 'src/main/java/com/baeldung/array/ArrayInitializer.java'. The issues are categorized by type (Code Smell, Vulnerability) and severity (Major, Minor, Info). The 'Vulnerability' filter is highlighted in the left sidebar, and the 'Major' severity filter is also highlighted. The list of issues includes:

- Add a private constructor to hide the implicit public one. (Code Smell, Major, Open, Not assigned, 30min effort)
- Move the array designator from the variable to the type. (Code Smell, Minor, Open, Not assigned, 5min effort)
- Move the array designator from the variable to the type. (Code Smell, Minor, Open, Not assigned, 5min effort)
- Move the array designator from the variable to the type. (Code Smell, Minor, Open, Not assigned, 5min effort)
- Immediately return this expression instead of assigning it to the temporary variable "array". (Code Smell, Minor, Open, Not assigned, 2min effort)
- Immediately return this expression instead of assigning it to the temporary variable "array". (Code Smell, Minor, Open, Not assigned, 2min effort)

El informe de SonarQube revela un proceso de desarrollo terrible (supuestamente): <https://www.baeldung.com/sonar-qube>



Herramientas dinámicas: DAST

Analizando el código tras su ejecución

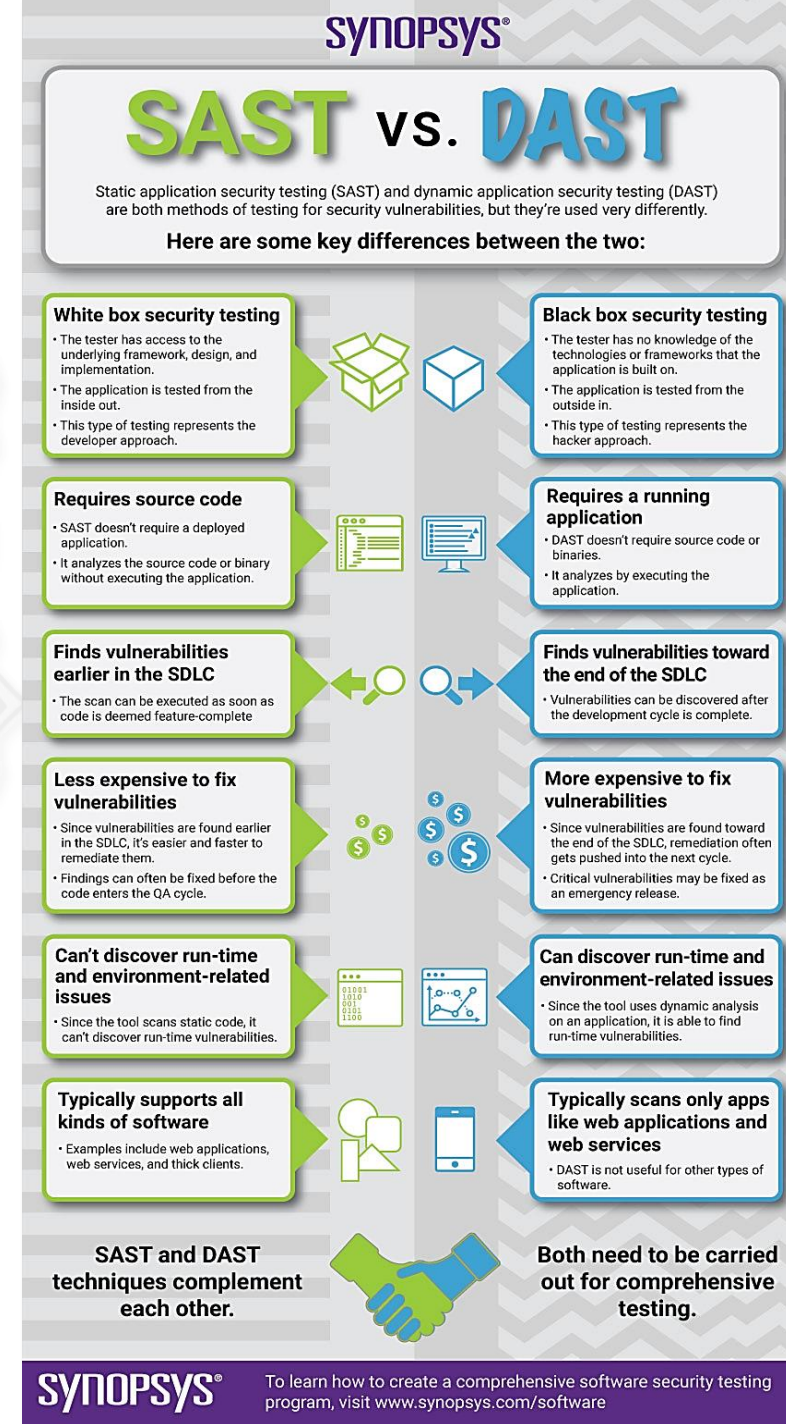


HERRAMIENTAS DAST (DYNAMIC APPLICATION SECURITY TESTING)



- Método de **prueba de caja negra** que introduce errores para probar las rutas de ejecución que estos provocan
 - No necesitan código fuente; **analiza la aplicación en ejecución**
 - **Ventajas**
 - A diferencia de SAST, permite detectar **problemas en ejecución**
 - Errores de autenticación y configuración de red, problemas tras login...
 - Hay menos falsos positivos
 - **Inconvenientes**
 - No dan información sobre las causas de las vulnerabilidades ni el código que las causa; ¿Por qué?
 - No adecuadas en las 1^{as} etapas del proyecto ¿Por qué?
 - No simula una parte de los posibles ataques

Fuente: <https://www.synopsys.com/blogs/software-security/sast-vs-dast-difference/>



HERRAMIENTAS DAST (DYNAMIC APPLICATION SECURITY TESTING)



● Una herramienta DAST muy popular es OWASP ZAP (en la imagen)

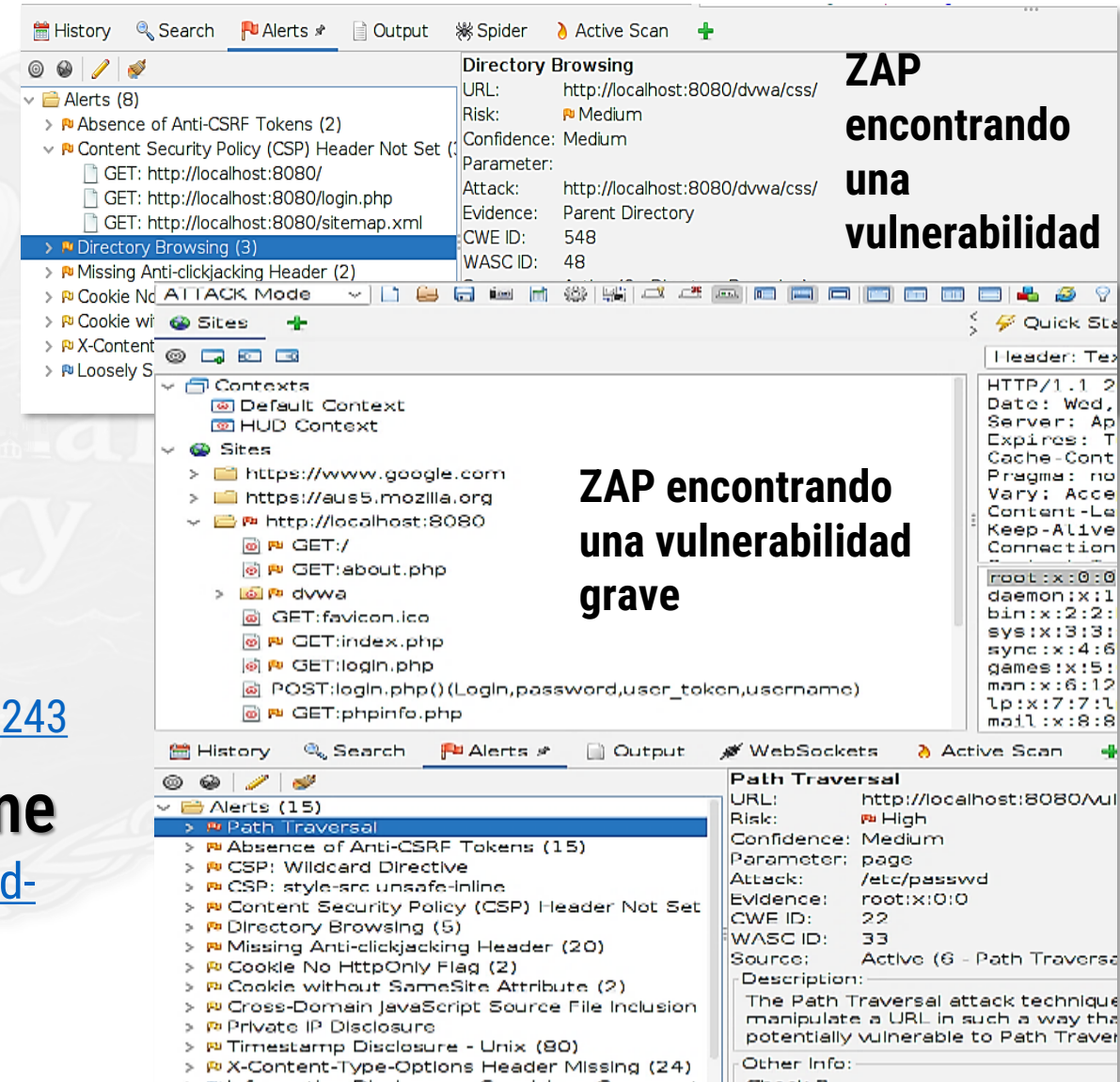
- <https://www.zaproxy.org/>
- Listado de estas herramientas
 - <https://www.comparitech.com/net-admin/dast-tools/>
- Algunas de ellas hacen más que solo DAST

● Manual de DAST sencillo con ZAP

- Sencillo y recomendable para tu TFG 😊
- <https://medium.com/@renbe/dast-with-owasp-zap-89fb5e1fa243>

● DAST integrado automatizado en un pipeline

- <https://www.everable.com/blog/automated-dast-in-ci/cd-pipeline-using-owasp-zap-a-comprehensive-guide>



ZAP
encontrando
una
vulnerabilidad

ZAP encontrando
una vulnerabilidad
grave

¿CREES QUE LO HAS ENTENDIDO TODO? ¡AUTOEVALÚATE!



● Eres capaz de entender lo siguiente

- *Que el desarrollo seguro requiere la integración de herramientas de testing de seguridad en la cadena de desarrollo*
- *Para qué sirve una herramienta SCA y el tipo de problemas que previene*
 - *Especialmente su labor crítica a la hora de identificar y prevenir CVEs de software de terceros que estemos usando*
- *Para qué sirve una herramienta SAST y el tipo de problemas que previene*
 - *Y que son capaces de decirnos la línea de código exacta donde detectó un problema*
 - *Además de cómo poner en marcha un SAST (y un SCA) muy fácilmente en GitHub*
- *Para qué sirve una herramienta DAST y el tipo de problemas que previene*
 - *Y que al final tratan a la aplicación como una "caja negra" en ejecución*

[< Ir al Índice](#)



DESPLIEGUE Y MANTENIMIENTO SEGUROS

Aspectos a tener en cuenta una vez la aplicación esté en producción



Departamento de Informática
Universidad de Oviedo



Universidad de Oviedo

¿QUÉ TE VAS A ENCONTRAR EN ESTE BLOQUE?



● En este bloque aprenderás

- Qué son las herramientas **WAF**
- Qué hay herramientas AST más avanzadas, como **RASP** o **UEBA**



● Al mantener nuestra aplicación desplegada, debemos considerar implementarla

- Lo que has aprendido en **ASR** sobre la **gestión** SS00 y nuestro **Tema 3**
 - Especialmente el **hardening** con los Benchmarks CIS
- **Monitorización continua**, registro y alertas de seguridad (**Tema 5**)
- **Procedimientos de respuesta a incidentes**
 - <https://www.incibe.es/empresas/tematicas/gestion-incidentes-seguridad>
- Usar **protección en tiempo de ejecución**
 - **Si desplegamos APIs: API Gateway**
 - Servicio que facilita crear, publicar, mantener, monitorizar y proteger una API
 - Actúan como la "puerta" para que las aplicaciones accedan a los datos, la lógica de negocio o la funcionalidad de tus servicios **backend**. Ej.: <https://aws.amazon.com/es/api-gateway/>
 - **Si desplegamos sitios web: Usa WAFs**



WEB APPLICATION FIREWALLS (WAFs)



- Módulos de servidor web que protegen sus aplicaciones alojadas contra ataques a nivel de aplicación

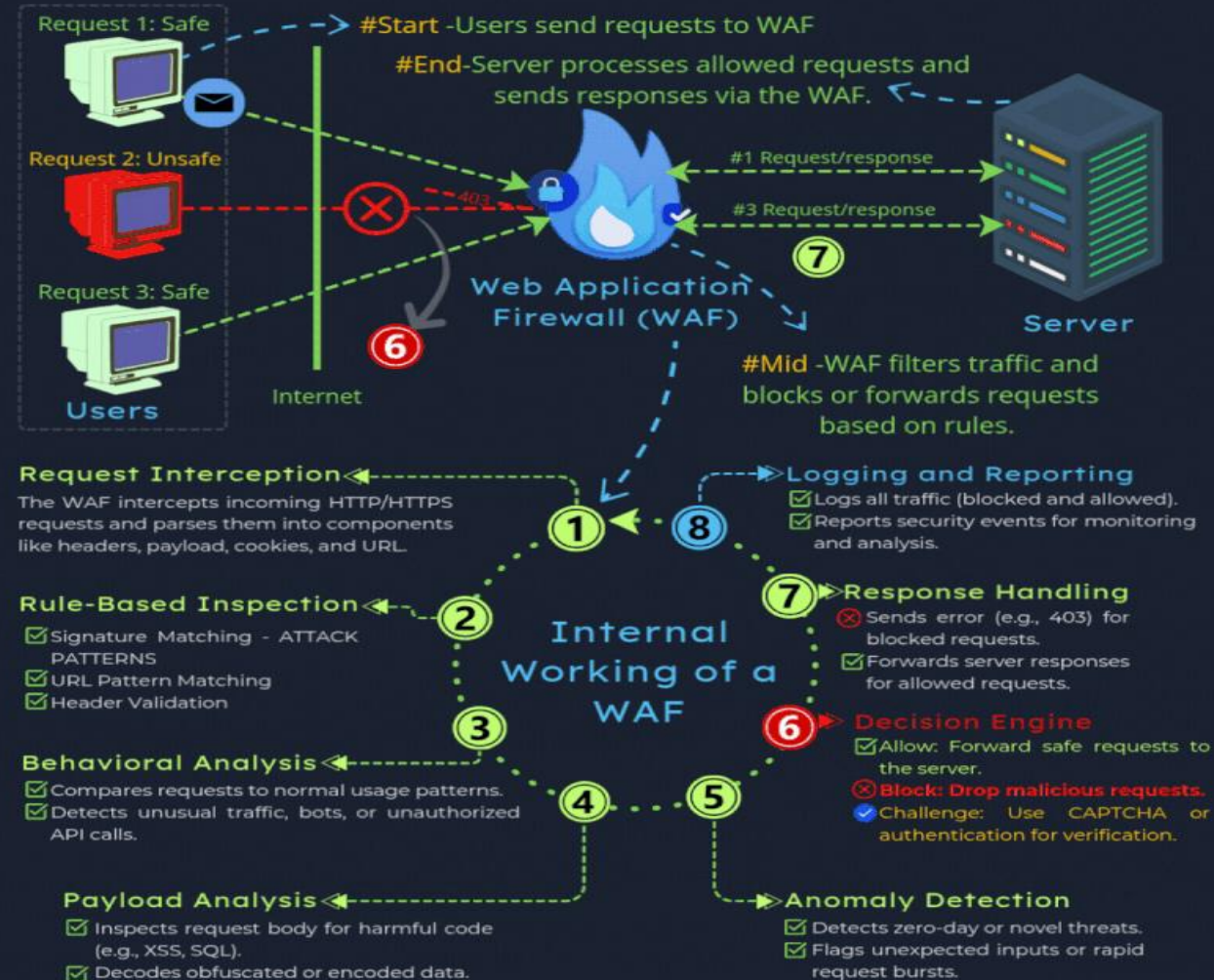
- **No despliegues sin uno**
- **mod_security** es probablemente el WAF gratuito más conocido (visto en labs)

- Proporcionan una capa adicional de protección automatizada

- Protección HTTP, listas de bloqueo de IPs en tiempo real, detección de ataques comunes, prevención de fugas accidentales de código fuente...

How WAF Works?

A **Web Application Firewall (WAF)** is a specialized security solution designed to monitor, filter, and protect HTTP and HTTPS traffic between a web application and the internet. It operates at the **application layer (Layer 7)** of the OSI model and is specifically tailored to detect and prevent attacks targeting web applications. A WAF can be network based, host based or cloud based.



OTRAS HERRAMIENTAS AST EN TIEMPO DE EJECUCIÓN



- Otras herramientas AST que podrías considerar para escenarios avanzados son
 - **IAST** (Interactive Application Security Testing): Instrumentan el software para evaluar su rendimiento y detectar vulnerabilidades
 - **Enfoque tipo agente:** agentes y sensores analizan continuamente el comportamiento de la aplicación
 - Durante las pruebas automatizadas/manuales o durante la ejecución
 - **RASP** (Runtime Application Self-Protection): Capaces de inspeccionar el comportamiento y el contexto de la aplicación
 - Captura todas las solicitudes verificando que son seguras y controla su validación dentro de la aplicación
 - Puede funcionar en modo **diagnóstico** (alerta) o **protección** (detener la aplicación, terminar la sesión...)
 - RASP e IAST tienen un uso similar, pero RASP no hace un análisis completo
 - **UEBA** (User and Entity Behavior Analytics): Herramientas de detección de anomalías con IA
 - **Analiza el comportamiento** de los usuarios, servidores, cuentas, portátiles, aplicaciones, etc.
 - ¡Todo está conectado a la red de una organización!
 - Se usan para detectar amenazas, fallos externos e intrusos
 - **Inside jobs, rogue users...** suponen un riesgo de seguridad creciente

¿CREEES QUE LO HAS ENTENDIDO TODO? ¡AUTOEVALÚATE!

?

● Eres capaz de entender lo siguiente

- *El concepto de API Gateway*
- *Qué es un WAF y cómo protege en el contexto del desarrollo de una aplicación web*
- *Qué es un RASP y cómo protege a una aplicación*
 - *Y que es un añadido que se sitúa dentro de la propia aplicación*
- *Qué es un UEBA y cómo protege a una aplicación*
 - *Y que realmente es una forma de detectar desviaciones en el comportamiento de usuarios y entidades*

- **Para leer más sobre DevSecOps**
 - <https://blog.pentesteracademy.com/devsecops-learning-path-integrating-security-with-devops-1cc03670552f?gi=b60a95ce4fcb>
- **Para una aproximación alternativa a DevSecOps, lee OWASP Proactive Controls**
 - <https://owasp.org/www-project-proactive-controls/>
- **Para trabajar con un pipeline Jenkins (usado en Arquitectura del Software, **ASW**) con las distintas herramientas de testing vistas**
 - Instalación de Jenkins en un contenedor Docker (adecuado para pruebas)
 - <https://www.jenkins.io/solutions/docker/>
 - Plugin Fortify para Jenkins (**SCA**)
 - <https://plugins.jenkins.io/fortify/>
 - Plugin Sonarqube para Jenkins (**SAST**) (también para otros productos similares)
 - <https://docs.sonarqube.org/latest/analysis/scan/sonarscanner-for-jenkins/>
 - **DAST** con OWASP ZAP y Jenkins
 - <https://digitalvarys.com/devsecops-dynamic-analysis-dast-with-owasp-zap-and-jenkins/>

TEMA 4 INTRODUCCIÓN A LA SEGURIDAD DE APLICACIONES



Departamento de Informática
Universidad de Oviedo



Universidad de Oviedo